



**2024 - 2025**

**PROJET DE FIN D'ÉTUDES**

**DIPLÔME NATIONAL D'INGÉNIEUR**

**SPÉCIALITÉ : INFORMATIQUE**

**CONCEPTION D'UN PIPELINE DEVSECOPS POUR  
SECURISER LES DEPLOIEMENTS A ENEO**

Réalisé par :

**Aurielle Clavel NJANEN NGANTCHUI**

Encadré par :

**A ESPRIT : M. Hassen JEDIDI**

**A Eneo : M. Thierry Fayell KUOBI NOUH**

**M. Franck Landry NSIMI MBIDA**



Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I  
validate the submission of the student's report:

- Nom & Prénom /Name & Surname : NIANEN NGANTCHU AURIELE CLAVEL

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : NDUH THIERRY FAYELL

Cachet & Signature / Stamp & Signature



Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : .....

Signature / Signature

Ce formulaire doit être rempli, signé et scanné/This form must be completed, signed and  
scanned.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover  
page.

# Dédicaces

*À mes parents,*

*Pour votre amour inconditionnel, vos sacrifices innombrables et le soutien indéfectible que vous m'avez toujours apporté. Ce travail est le fruit de l'éducation que vous m'avez transmise,*

*À mes sœurs et à toute ma famille,*

*Pour votre présence, vos encouragements constants et la chaleur de votre affection qui ont été ma source de motivation tout au long de ce parcours,*

*À tous ceux qui croient en l'excellence,  
ce mémoire vous est dédié.*

# Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à son aboutissement et à la réussite de mon stage.

Mes remerciements vont tout d'abord à **Monsieur le Directeur Général d'Eneo Cameroon S.A.**, pour m'avoir accueilli au sein de cette structure de référence et pour les moyens mis à ma disposition.

Je tiens à témoigner ma reconnaissance à Messieurs **Abderrazek HACHANI** et **David MAGOUDAYA**, dont l'intervention et la confiance ont été déterminantes pour l'obtention de ce stage. Merci de m'avoir ouvert les portes de cette opportunité professionnelle.

J'exprime ma profonde gratitude à mon encadrant académique académique, **M. Hassen JEDIDI** et à l'expert académique **Mme Hajer BARGAOUI**, pour ses conseils avisés, sa disponibilité et la rigueur scientifique qu'ils ont su m'inculquer tout au long de la rédaction de ce mémoire.

Je remercie chaleureusement mes encadrants professionnels pour leur accompagnement quotidien :

- À mon mentor, **M. Thierry Fayell KUOBI**, pour son partage d'expertise technique, sa pédagogie et sa vision qui ont grandement enrichi mes compétences en DevSecOps.
- À mon accompagnateur, **M. Franck Landry MBIDA**, pour son suivi, ses orientations et son soutien constant au sein de l'entreprise.

Ma reconnaissance s'adresse également à l'ensemble des membres du **département IRM (Information Risk Management)** et de la **BSOF**, pour leur accueil chaleureux, leur esprit de collaboration et l'environnement de travail stimulant qu'ils m'ont offert.

Enfin, je remercie tous mes amis, proches et camarades qui, de près ou de loin, m'ont soutenu moralement et encouragé durant cette période.

# Résumé

Ce mémoire présente la conception et le déploiement du socle fondateur d'une démarche DevSecOps au sein de l'unité Information Risk Management (IRM) d'Eneo Cameroon S.A.

L'analyse initiale de l'infrastructure révèle un défi majeur : un environnement critique mais techniquement cloisonné, caractérisé par des déploiements manuels, l'absence de chaîne d'intégration continue (CI/CD) et une visibilité limitée sur la sécurité applicative réelle. Dans ce contexte de maturité DevOps naissante et sous de fortes contraintes budgétaires, l'implémentation immédiate d'un écosystème complexe s'avérerait inadaptée.

Ce projet se concentre donc sur la mise en œuvre pragmatique du « Noyau Opérationnel » de la sécurité : un pipeline automatisé et une tour de contrôle centralisée. S'appuyant sur l'approche Secure by Design, l'architecture orchestrée par **Jenkins** et **Docker** intègre les contrôles de sécurité essentiels (SAST avec **SonarQube**, SCA avec **Snyk**, Sécurité des Conteneurs avec **Trivy**) et centralise les données via l'**Elastic Stack**.

Le résultat est une plateforme de référence qui transforme un processus manuel opaque en un flux mesurable. La solution permet désormais de détecter les vulnérabilités avant la production et offre, grâce aux tableaux de bord d'observabilité, une vision claire du niveau de risque. L'analyse des métriques (MTTR) valide la capacité du dispositif à réduire les délais de remédiation, fournissant ainsi à Eneo les fondations indispensables pour l'évolution future de sa posture de sécurité.

**Mots-clés :** DevSecOps, Pipeline CI/CD, Observabilité, Automatisation, Jenkins, Elastic Stack, Gestion des Vulnérabilités, Secure by Design.

# Table des matières

|                                                                        |             |
|------------------------------------------------------------------------|-------------|
| <b>Dédicaces</b>                                                       | <b>iii</b>  |
| <b>Remerciements</b>                                                   | <b>iv</b>   |
| <b>Résumé</b>                                                          | <b>v</b>    |
| <b>Liste des figures</b>                                               | <b>xi</b>   |
| <b>Liste des tableaux</b>                                              | <b>xii</b>  |
| <b>Liste des Abréviations</b>                                          | <b>xiii</b> |
| <b>Introduction Générale</b>                                           | <b>1</b>    |
| <b>1 Présentation Générale et Cadrage du Projet</b>                    | <b>3</b>    |
| 1.1 Introduction . . . . .                                             | 3           |
| 1.2 Contexte du Stage chez Eneo . . . . .                              | 3           |
| 1.2.1 Présentation de l'Entreprise d'Accueil . . . . .                 | 3           |
| 1.2.2 L'unité IRM (Information Risk Management) . . . . .              | 4           |
| 1.3 Cadre du projet . . . . .                                          | 4           |
| 1.3.1 Contexte Opérationnel et Technique d'Eneo . . . . .              | 5           |
| 1.3.2 Identification des Freins à l'Adoption . . . . .                 | 5           |
| 1.3.3 Analyse des Contraintes du Projet . . . . .                      | 6           |
| 1.4 Méthodologie de Travail . . . . .                                  | 7           |
| 1.5 Conclusion . . . . .                                               | 8           |
| <b>2 Fondements Théoriques et État de l'Art</b>                        | <b>9</b>    |
| 2.1 Introduction . . . . .                                             | 9           |
| 2.2 Fondements Théoriques du DevSecOps . . . . .                       | 9           |
| 2.2.1 Du DevOps au DevSecOps : Évolution Nécessaire . . . . .          | 9           |
| 2.2.2 Sécurité par Conception ("Secure by Design") . . . . .           | 10          |
| 2.2.3 Culture de la Collaboration et Responsabilité Partagée . . . . . | 10          |
| 2.2.4 Automatisation et Amélioration Continue . . . . .                | 10          |
| 2.2.5 Le Principe "Fail Fast, Fail Early" . . . . .                    | 10          |

## TABLE DES MATIÈRES

---

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| 2.3      | Étude de l'Art : Retour d'expérience des acteurs africains . . . . .       | 11        |
| 2.3.1    | L'Impératif DevSecOps et la Transformation des Acteurs Majeurs . . . . .   | 11        |
| 2.3.2    | Tableau Comparatif des Stratégies DevSecOps . . . . .                      | 13        |
| 2.3.3    | Conclusion du Chapitre . . . . .                                           | 14        |
| <b>3</b> | <b>Conception de la Solution et Architecture Technique</b>                 | <b>16</b> |
| 3.1      | Introduction . . . . .                                                     | 16        |
| 3.1.1    | Héritage de l'État de l'Art et Justification du Design . . . . .           | 16        |
| 3.2      | Démarche de Conception : Le Secure by Design en Pratique . . . . .         | 17        |
| 3.3      | Solution Proposée . . . . .                                                | 17        |
| 3.4      | Spécifications et Cahier des Charges de la Solution . . . . .              | 18        |
| 3.4.1    | Synthèse des Exigences Issues de l'État de l'Art . . . . .                 | 18        |
| 3.5      | Choix Technologiques Pour la Solution . . . . .                            | 18        |
| 3.5.1    | Validation et Complétion de la Stack Technologique . . . . .               | 19        |
| 3.5.2    | Orchestrateur CI/CD (Exigence E1) . . . . .                                | 19        |
| 3.5.3    | Analyse de Sécurité Statique – SAST (Exigence E2) . . . . .                | 20        |
| 3.5.4    | Analyse de Composition Logicielle – SCA (Exigence E2) . . . . .            | 22        |
| 3.5.5    | Analyse de Vulnérabilités des Conteneurs (Exigence E2) . . . . .           | 23        |
| 3.5.6    | Plateforme d'Observabilité (Exigence E3 & E4) . . . . .                    | 24        |
| 3.5.7    | Technologie de Conteneurisation (Exigence E5) . . . . .                    | 26        |
| 3.5.8    | Technologie de Gestion de Code Source (SCM) . . . . .                      | 27        |
| 3.6      | Architecture Globale Cible . . . . .                                       | 28        |
| 3.6.1    | Le Flux d'Orchestration et les Scans de Sécurité . . . . .                 | 29        |
| 3.6.2    | L'Unification des Données et l'Observabilité . . . . .                     | 29        |
| 3.6.3    | Le Système de Réponse et de Mesure (MTTD/MTTR) . . . . .                   | 30        |
| 3.6.4    | Valeur Ajoutée et Spécificité de la Solution Proposée . . . . .            | 30        |
| 3.7      | Conclusion . . . . .                                                       | 31        |
| <b>4</b> | <b>Mise en Œuvre de l'Infrastructure de Base et Stabilisation des Flux</b> | <b>32</b> |
| 4.1      | Introduction . . . . .                                                     | 32        |
| 4.2      | Le Cœur de l'Infrastructure : Docker Compose . . . . .                     | 32        |
| 4.2.1    | Présentation du fichier d'orchestration . . . . .                          | 33        |
| 4.2.2    | Configuration des Services Clés . . . . .                                  | 33        |
| 4.2.3    | Le Réseau Docker devnet . . . . .                                          | 34        |
| 4.2.4    | Gestion de la Persistance des Données . . . . .                            | 34        |
| 4.3      | Préparation de l'Agent Jenkins Personnalisé . . . . .                      | 35        |
| 4.3.1    | Justification des Choix Techniques . . . . .                               | 35        |
| 4.4      | Configuration de l'Intégration GitLab-Jenkins . . . . .                    | 36        |
| 4.4.1    | Génération et Échange de Clés SSH . . . . .                                | 36        |
| 4.4.2    | Configuration des Webhooks GitLab . . . . .                                | 36        |

## TABLE DES MATIÈRES

---

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 4.4.3    | Gestion des Credentials Jenkins . . . . .                            | 36        |
| 4.5      | Validation du Flux Initial : Premier Pipeline . . . . .              | 37        |
| 4.5.1    | Structure du Pipeline . . . . .                                      | 37        |
| 4.5.2    | Capture des Timestamps et Métadonnées Git . . . . .                  | 37        |
| 4.5.3    | Exécution des Analyses de Sécurité . . . . .                         | 37        |
| 4.5.4    | Architecture du Répertoire Partagé . . . . .                         | 37        |
| 4.5.5    | Génération des Rapports Bruts . . . . .                              | 37        |
| 4.6      | Résolution des Noms de Domaine Locaux . . . . .                      | 38        |
| 4.7      | Conclusion . . . . .                                                 | 38        |
| <b>5</b> | <b>Mise en Œuvre de l’Observabilité</b>                              | <b>39</b> |
| 5.1      | Introduction : Au-delà des Logs de Sécurité . . . . .                | 39        |
| 5.2      | Pilier 1 : Normalisation et Collecte des Logs de Sécurité . . . . .  | 39        |
| 5.2.1    | Normalisation et Splitting (Le Cœur) . . . . .                       | 40        |
| 5.2.2    | Collecte (Filebeat) . . . . .                                        | 43        |
| 5.3      | Pilier 2 : Métriques d’Infrastructure (Metricbeat) . . . . .         | 44        |
| 5.3.1    | Configuration Stratégique des Modules . . . . .                      | 44        |
| 5.3.2    | Enrichissement Contextuel . . . . .                                  | 44        |
| 5.4      | Pilier 3 : Traces Applicatives et RUM (Elastic APM) . . . . .        | 44        |
| 5.4.1    | Instrumentation du Backend (Java Agent) . . . . .                    | 44        |
| 5.4.2    | Instrumentation du Frontend (RUM Agent) . . . . .                    | 45        |
| 5.4.3    | La Continuité de la Trace (Distributed Tracing) . . . . .            | 45        |
| 5.4.4    | Suivi des Parcours Utilisateur (Custom Transactions) . . . . .       | 45        |
| 5.5      | Ingestion, Stockage et Indexation (Elasticsearch) . . . . .          | 45        |
| 5.5.1    | Les Ingest Pipelines (Le Cerveau de l’Ingestion) . . . . .           | 45        |
| 5.6      | Ingestion, Stockage et Indexation (Elasticsearch) . . . . .          | 46        |
| 5.6.1    | Ingest Pipelines . . . . .                                           | 46        |
| 5.6.2    | Template, Mapping et Indexation . . . . .                            | 46        |
| 5.6.3    | Le rôle du champ <code>doc_type</code> . . . . .                     | 47        |
| 5.7      | Conclusion . . . . .                                                 | 47        |
| <b>6</b> | <b>Observabilité et Visualisation : La Tour de Contrôle</b>          | <b>49</b> |
| 6.1      | Introduction . . . . .                                               | 49        |
| 6.2      | Dashboard 1 : Security Unified . . . . .                             | 49        |
| 6.2.1    | Présentation . . . . .                                               | 49        |
| 6.2.2    | Visualisations . . . . .                                             | 50        |
| 6.3      | Dashboard 2 : Snyk SCA (Analyse de Composition Logicielle) . . . . . | 51        |
| 6.3.1    | Présentation . . . . .                                               | 51        |
| 6.3.2    | Visualisations . . . . .                                             | 52        |
| 6.4      | Dashboard 3 : Trivy Container Security . . . . .                     | 53        |

## TABLE DES MATIÈRES

---

|          |                                                                                                                      |           |
|----------|----------------------------------------------------------------------------------------------------------------------|-----------|
| 6.4.1    | Présentation . . . . .                                                                                               | 53        |
| 6.4.2    | Visualisations . . . . .                                                                                             | 53        |
| 6.5      | Dashboard 4 : SonarQube SAST + Qualité . . . . .                                                                     | 54        |
| 6.5.1    | Présentation . . . . .                                                                                               | 54        |
| 6.5.2    | Visualisations . . . . .                                                                                             | 54        |
| 6.6      | Dashboard 5 : Infrastructure (Metricbeat) . . . . .                                                                  | 55        |
| 6.6.1    | Présentation . . . . .                                                                                               | 55        |
| 6.6.2    | Visualisations . . . . .                                                                                             | 55        |
| 6.7      | Dashboard 6 : Performance Applicative (APM/RUM) . . . . .                                                            | 56        |
| 6.7.1    | Présentation . . . . .                                                                                               | 56        |
| 6.7.2    | Visualisations . . . . .                                                                                             | 56        |
| 6.8      | Conclusion . . . . .                                                                                                 | 56        |
| <b>7</b> | <b>Exploitation Opérationnelle de l'Observabilité : Automatisation du Suivi des Vulnérabilités et Calcul du MTTR</b> | <b>57</b> |
| 7.1      | Introduction . . . . .                                                                                               | 57        |
| 7.2      | Watcher 1 : watcher_mark_resolved (Le Calculateur de Métriques) . . . . .                                            | 59        |
| 7.2.1    | Logique du Code . . . . .                                                                                            | 59        |
| 7.2.2    | Fonctionnalités Techniques . . . . .                                                                                 | 60        |
| 7.3      | Watcher 2 : vulnerability_report_watcher (Le Notificateur Slack) . . . . .                                           | 60        |
| 7.3.1    | Logique du Code . . . . .                                                                                            | 61        |
| 7.3.2    | Fonctionnalités Techniques . . . . .                                                                                 | 62        |
| 7.4      | Analyse des Métriques MTTR : Résultats et Interprétation . . . . .                                                   | 62        |
| 7.4.1    | Méthodologie . . . . .                                                                                               | 62        |
| 7.4.2    | Résultats observés . . . . .                                                                                         | 62        |
| 7.4.3    | Contexte du Proof of Concept (POC) . . . . .                                                                         | 63        |
| 7.5      | Conclusion . . . . .                                                                                                 | 64        |
|          | <b>Conclusion Générale</b>                                                                                           | <b>65</b> |
|          | <b>Bibliographie et Webographie</b>                                                                                  | <b>67</b> |
|          | <b>Annexes</b>                                                                                                       | <b>70</b> |
| <b>A</b> | <b>Détails de Configuration et Scripts (Chapitre 4)</b>                                                              | <b>71</b> |
| A.1      | Configurations du fichier docker-compose.yml . . . . .                                                               | 71        |
| A.1.1    | Configuration du service GitLab . . . . .                                                                            | 71        |
| A.1.2    | Configuration du service Jenkins . . . . .                                                                           | 71        |
| A.1.3    | Configuration SonarQube et PostgreSQL . . . . .                                                                      | 72        |
| A.1.4    | Configuration Elastic Stack (Elasticsearch et Kibana) . . . . .                                                      | 73        |
| A.1.5    | Configuration Filebeat . . . . .                                                                                     | 73        |

## TABLE DES MATIÈRES

---

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| A.2      | Construction de l'Agent Jenkins Personnalisé . . . . .                      | 74        |
| A.2.1    | Dockerfile de l'Agent Jenkins . . . . .                                     | 74        |
| A.3      | Commandes et Scripts de Configuration . . . . .                             | 75        |
| A.3.1    | Commandes de déploiement du réseau et des services . . . . .                | 75        |
| A.3.2    | Génération des clés SSH pour Jenkins . . . . .                              | 75        |
| A.3.3    | Test de connexion SSH Jenkins vers GitLab . . . . .                         | 76        |
| A.3.4    | Configuration DNS locale (/etc/hosts) . . . . .                             | 76        |
| A.4      | Scripts du Pipeline CI/CD . . . . .                                         | 76        |
| A.4.1    | Structure globale du Jenkinsfile . . . . .                                  | 76        |
| A.4.2    | Stage 1 : Récupération et Métadonnées (Capture T0) . . . . .                | 77        |
| A.4.3    | Stage SAST : Analyse SonarQube Backend . . . . .                            | 77        |
| A.4.4    | Authentification Snyk et Docker . . . . .                                   | 78        |
| <b>B</b> | <b>Détails d'Implémentation de l'Observabilité (Chapitre 5)</b>             | <b>79</b> |
| B.1      | Configuration Metricbeat (Docker & Processors) . . . . .                    | 79        |
| B.2      | Injection de l'Agent Java (Backend) . . . . .                               | 79        |
| B.3      | Initialisation RUM (Frontend) . . . . .                                     | 80        |
| B.4      | Intercepteur HTTP pour le Tracing Distribué . . . . .                       | 80        |
| B.5      | Instrumentation de la Navigation (Custom Transaction) . . . . .             | 80        |
| <b>C</b> | <b>Détails de l'automatisation du suivi de l'observabilité (Chapitre 7)</b> | <b>82</b> |
| C.1      | Code Watcher : Détection et Résolution des Vulnérabilités . . . . .         | 82        |
| C.2      | Code de la Requête MTTR . . . . .                                           | 91        |

# Table des figures

|     |                                                                                   |    |
|-----|-----------------------------------------------------------------------------------|----|
| 1.1 | Logo de l'entreprise Eneo Cameroon S.A. . . . . .                                 | 3  |
| 1.2 | Structure organisationnelle de l'unité IRM . . . . .                              | 4  |
| 2.1 | Le cycle de vie DevOps sans intégration de la sécurité . . . . .                  | 9  |
| 2.2 | Le principe du "Shift Left" : intégrer la sécurité dès la conception . . . . .    | 10 |
| 3.1 | Architecture Globale DevSecOps Cible . . . . .                                    | 29 |
| 5.1 | Flux de données de la génération des rapports bruts à l'ingestion Elasticsearch . | 40 |
| 6.1 | Dashboard Security Unified - Vue globale de la posture de sécurité . . . . .      | 50 |
| 6.2 | Dashboard Security Unified - Vue globale de la posture de sécurité . . . . .      | 51 |
| 6.3 | Dashboard Snyk SCA - Analyse des dépendances . . . . .                            | 52 |
| 6.4 | Dashboard Trivy Container Security - Sécurité des images Docker . . . . .         | 53 |
| 6.5 | Dashboard SonarQube SAST + Qualité - Santé du code . . . . .                      | 54 |
| 6.6 | Dashboard Metricbeat Infrastructure - Signes vitaux des conteneurs . . . . .      | 55 |
| 6.7 | Expérience utilisateur détaillée . . . . .                                        | 56 |
| 7.1 | Architecture logique du système d'alerting et de calcul MTTR . . . . .            | 58 |
| 7.2 | Notifications Slack - Canal Security . . . . .                                    | 61 |
| 7.3 | Évolution du Temps Moyen de Résolution (MTTR) sur les trois jours actifs . .      | 63 |

# Liste des tableaux

|     |                                                                             |    |
|-----|-----------------------------------------------------------------------------|----|
| 1.1 | Synthèse de l'étude de l'existant chez Eneo . . . . .                       | 5  |
| 1.2 | Risques identifiés dans l'environnement existant . . . . .                  | 6  |
| 1.3 | Contraintes du projet . . . . .                                             | 7  |
| 2.1 | Tableau comparatif des stratégies DevSecOps des acteurs africains . . . . . | 14 |
| 3.1 | Cahier des charges de la solution . . . . .                                 | 18 |
| 3.2 | Comparaison des orchestrateurs CI/CD . . . . .                              | 19 |
| 3.3 | Comparaison des outils SAST . . . . .                                       | 21 |
| 3.4 | Comparaison des outils SCA . . . . .                                        | 22 |
| 3.5 | Comparaison des outils de scan de conteneurs . . . . .                      | 23 |
| 3.6 | Comparaison des plateformes d'observabilité . . . . .                       | 24 |
| 3.7 | Comparaison des technologies de conteneurisation . . . . .                  | 27 |
| 3.8 | Comparaison des systèmes de gestion de code source . . . . .                | 28 |
| 4.1 | Répartition des services Docker Compose . . . . .                           | 33 |
| 4.2 | Volumes persistants de l'infrastructure . . . . .                           | 35 |
| 4.3 | Volumes persistants de l'infrastructure . . . . .                           | 35 |
| 4.4 | Credentials nécessaires pour le pipeline . . . . .                          | 36 |
| 4.5 | Exemple de rapports bruts générés . . . . .                                 | 38 |
| 5.1 | Rôles des composants Filebeat . . . . .                                     | 43 |
| 5.2 | Rôles des composants de l'architecture d'ingestion Elasticsearch . . . . .  | 47 |
| 7.1 | Termes Clés et Définitions (Watcher 1) . . . . .                            | 60 |
| 7.2 | Termes Clés et Définitions (Watcher 2) . . . . .                            | 62 |
| 7.3 | Répartition temporelle des résolutions et MTTR moyen . . . . .              | 62 |

# Liste des Abréviations

|                  |                                                |
|------------------|------------------------------------------------|
| <b>APM</b>       | Application Performance Monitoring             |
| <b>CI/CD</b>     | Continuous Integration / Continuous Deployment |
| <b>CVE</b>       | Common Vulnerabilities and Exposures           |
| <b>DAST</b>      | Dynamic Application Security Testing           |
| <b>DevSecOps</b> | Development, Security and Operations           |
| <b>IRM</b>       | Information Risk Management                    |
| <b>MTTD</b>      | Mean Time To Detect                            |
| <b>MTTR</b>      | Mean Time To Resolve                           |
| <b>SAST</b>      | Static Application Security Testing            |
| <b>SCA</b>       | Software Composition Analysis                  |

# Introduction Générale

Le secteur de l'énergie au Cameroun, opéré par Eneo Cameroon S.A., est une cible critique. La numérisation croissante des services expose l'entreprise à des cybermenaces sophistiquées. Pour répondre à ces défis, Eneo a investi dans des technologies de protection périmétrique. Cependant, la sécurité applicative reste un axe d'amélioration majeur.

Le processus de livraison logicielle actuel, bien que fonctionnel, repose encore largement sur des interventions manuelles. Il n'existe pas, à ce jour, de mécanisme standardisé et automatisé pour évaluer la sécurité du code produit ou acquis avant sa mise en production. L'équipe IRM (Information Risk Management) fait face à un défi de visibilité : sans automatisation, il est difficile de garantir qu'une application est sécurisée sans ralentir les opérations de déploiement.

C'est dans ce contexte de transition que s'inscrit ce projet de fin d'études. Le défi ne consiste pas simplement à déployer des outils de sécurité, mais à introduire une culture de l'automatisation et de la mesure dans un environnement qui initialise sa démarche DevOps.

## Problématique

Face à un environnement dépourvu de chaîne d'intégration continue automatisée, la question centrale qui oriente ce travail est la suivante :

**« Comment concevoir et déployer le socle technique d'une démarche DevSecOps permettant d'automatiser les contrôles de sécurité essentiels et d'offrir une visibilité immédiate sur les risques, dans un contexte organisationnel à faible maturité DevOps ? »**

Pour répondre à cette problématique, le projet se structure autour de trois objectifs opérationnels :

1. **Construire le pipeline d'intégration** : Mettre en place l'automatisation manquante (CI/CD) pour permettre l'exécution systématique des tests.
2. **Intégrer les contrôles fondamentaux (Shift Left)** : Automatiser l'analyse du code (SAST), des dépendances (SCA) et des conteneurs pour assainir les bases logicielles.
3. **Garantir l'observabilité** : Centraliser les résultats dans une « Tour de Contrôle » unifiée pour permettre aux équipes de piloter la sécurité par la donnée.

Ce mémoire retrace la construction de cette plateforme, depuis l'analyse des besoins jusqu'à la mesure de son efficacité. Il s'articule en **huit chapitres** :

- **Chapitre 1** : Contexte, analyse de l'existant, problématique
- **Chapitre 2** : Fondements théoriques et état de l'art

- **Chapitre 3** : Conception de la Solution et Architecture Technique
- **Chapitre 4** : Mise en Œuvre de l'Infrastructure de Base et Stabilisation des Flux
- **Chapitre 5** : Mise en Œuvre de l'Observabilité
- **Chapitre 6** : Observabilité et Visualisation : La Tour de Contrôle
- **Chapitre 7** : Exploitation Opérationnelle de l'Observabilité : Automatisation du Suivi des Vulnérabilités et Calcul du MTTR

# Chapitre 1

## Présentation Générale et Cadrage du Projet

### 1.1 Introduction

Toute démarche d'ingénierie commence par une analyse rigoureuse. Avant de construire une solution, il est impératif de comprendre le contexte, les problèmes à résoudre et les contraintes. Ce chapitre présente l'entreprise d'accueil Eneo, dresse un état des lieux des pratiques existantes, identifie les risques critiques et formalise la problématique qui a guidé l'intégralité du projet.

### 1.2 Contexte du Stage chez Eneo

#### 1.2.1 Présentation de l'Entreprise d'Accueil

**Eneo Cameroon S.A.** est l'opérateur historique du secteur de l'électricité au Cameroun, assurant la production, le transport et la distribution d'énergie électrique sur l'ensemble du territoire national. Avec plus de **1,2 million de clients** répartis dans 10 régions, Eneo joue un rôle stratégique dans le développement économique du pays.



FIGURE 1.1 – Logo de l'entreprise Eneo Cameroon S.A.

En tant qu'acteur d'un secteur d'importance vitale, la robustesse, la fiabilité et la **sécurité** de ses systèmes d'information constituent des enjeux stratégiques majeurs. Toute défaillance peut avoir des conséquences directes sur la continuité de service et la satisfaction client.

### 1.2.2 L'unité IRM (Information Risk Management)

Mon stage s'est déroulé au sein de l'unité **IRM (Information Risk Management)**, rattachée à la Direction de l'Innovation des Systèmes d'Information (DISI). La mission de l'IRM est de garantir la **confidentialité, l'intégrité et la disponibilité** des informations de l'entreprise. L'unité IRM se subdivise en deux pôles complémentaires :

- **Governance** : Définition des politiques de sécurité, gestion des risques informatiques, conformité réglementaire
- **Cyber Security** : Surveillance des infrastructures, détection d'incidents, réponse aux menaces, audits de sécurité

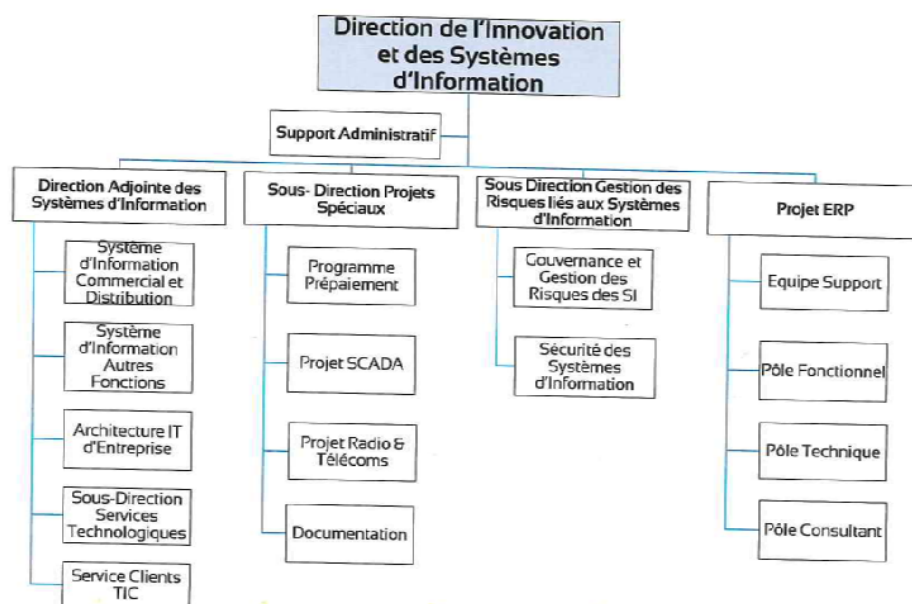


FIGURE 1.2 – Structure organisationnelle de l'unité IRM

C'est dans ce contexte, à la croisée des chemins entre la sécurité (Sec) et les opérations (Ops), que le besoin d'une meilleure intégration avec les équipes de développement (Dev) est apparu comme une **nécessité stratégique**.

## 1.3 Cadre du projet

### 1.3.1 Contexte Opérationnel et Technique d'Eneo

L'audit technique réalisé en début de projet a mis en lumière un écart entre les ambitions de sécurité et la réalité opérationnelle. Bien que des outils comme GitLab soient présents, leur utilisation reste limitée au stockage de code, sans exploitation des capacités d'intégration continue (CI). Les déploiements manuels créent un manque de visibilité sur la qualité réelle du code mis en production. Le tableau 1.1 synthétise les observations majeures.

| Domaine             | Situation Actuelle                                                                                                             |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Orchestration CI/CD | <b>Inexistante.</b> Les développeurs poussent manuellement vers la production.                                                 |
| Déploiement         | <b>Manuel.</b> Effectué par le développeur concerné.                                                                           |
| Conteneurisation    | <b>Non utilisée.</b> Applications déployées directement sur les serveurs.                                                      |
| Qualité du Code     | <b>Aucune analyse</b> prouvant que le code source est fiable, efficace et facile à maintenir. Tests unitaires non généralisés. |
| Sécurité (Code)     | <b>Faible.</b> Mots de passe faible en clair, absence de contrôle de sécurité applicative.                                     |
| Tests               | <b>Manuels</b> (Postman/Swagger).                                                                                              |
| Documentation       | <b>Inexistante.</b> Transmission orale ou individuelle, pas de base documentaire.                                              |

TABLE 1.1 – Synthèse de l'étude de l'existant chez Eneo

### 1.3.2 Identification des Freins à l'Adoption

L'audit interne a permis d'établir une série de **cinq objectifs de transformation** (O1 à O5) que l'entreprise doit adresser pour renforcer sa posture de sécurité. Ces objectifs, bien qu'ambitieux, servent de feuille de route au projet. Ils sont résumés dans le tableau 1.2 ci-dessous :

| ID | Observation                                        | Impact Potentiel                                                                                                                        |
|----|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| O1 | Processus de déploiement manuel                    | Perte de traçabilité, erreurs humaines, lenteur des livraisons                                                                          |
| O2 | Absence de contrôle de qualité et sécurité du code | Vulnérabilités non détectées, dette technique croissante                                                                                |
| O3 | Manque de gouvernance des outils de sécurité       | Rapports dispersés, outils de supervision non audités, rendant l’audit difficile.                                                       |
| O4 | Faible réactivité et MTTR élevé                    | Aucun audit systématique ni documentation centralisée des vulnérabilités, impossibilité de mesurer le Temps Moyen de Remédiation (MTTR) |
| O5 | Équipe Sécurité agit en fin de cycle               | Corrections tardives, goulot d’étranglement (anti-DevOps)                                                                               |

TABLE 1.2 – Risques identifiés dans l’environnement existant

### 1.3.3 Analyse des Contraintes du Projet

La solution doit s’insérer dans le contexte spécifique d’Eneo. Le tableau 1.3 détaille les contraintes majeures.

| Type              | Contrainte                                               | Impact sur le Projet                                                                                                                                                            |
|-------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Budgétaire        | Utilisation exclusive d'outils open-source               | Exclusion de solutions SaaS propriétaires. Choix plus orienté vers le modèle "community + Enterprise" permettant une montée en gamme progressive si le PoC est concluant        |
| Organisationnelle | Absence d'équipe DevOps dédiée, culture en silo          | Solution doit être la plus automatisée et simple à maintenir pour servir de PoC convaincant.                                                                                    |
| Technique         | Pas d'infrastructure CI/CD ni Docker existante           | Solution doit être bâtie de zéro ( <i>from scratch</i> ).                                                                                                                       |
| Humaine           | Développeurs peu sensibilisés à la sécurité              | Solution doit être pédagogique. Rapports et guides essentiels.                                                                                                                  |
| Déploiement       | Infrastructure de production non allouée au stade du PoC | La solution sera d'abord validée sur une VM personnelle en environnement Docker Compose. En cas de succès, un plan de migration vers le datacenter interne d'Eneo sera élaboré. |

TABLE 1.3 – Contraintes du projet

### 1.4 Méthodologie de Travail

Pour mener à bien ce projet, j'ai adopté le **Secure SDLC** une approche méthodologique dont les activités et phases clés ont été structurées en **quatre grandes phases**, alignée sur les principes DevSecOps d'itération, d'automatisation progressive et de mesure continue :

#### 1. Phase 1 : Analyse de l'Existant & Conception de l'Architecture

Audit de l'environnement applicatif et infrastructure d'Eneo, identification des besoins de sécurité, et conception de l'architecture DevSecOps cible.

#### 2. Phase 2 : Construction du Pipeline CI/CD Sécurisé

Mise en place de l'infrastructure de base et implémentation du pipeline intégrant les contrôles de sécurité automatisés : analyse statique du code (SAST), détection des vulnérabilités dans les dépendances (SCA), et scan des images de conteneurs. Validation du flux de données vers la plateforme d'observabilité.

#### 3. Phase 3 : Observabilité

Conception d'une vue consolidée de l'état de la sécurité à travers la création de tableaux de bord thématiques. Cette phase vise à transformer les données brutes en indicateurs exploitables pour faciliter la compréhension, la priorisation et le pilotage des vulnérabilités par les équipes.

### 4. Phase 4 : Automatisation du Suivi & Mesure de la Performance

Développement des mécanismes permettant l'exploitation opérationnelle des données issues de l'observabilité : automatisation de l'identification des vulnérabilités résolues, calcul continu des métriques clefs (dont le MTTR) et génération d'alertes pertinentes afin d'optimiser la boucle de feedback et d'améliorer la réactivité des équipes.

Chaque phase s'est déroulée sur une période de **3 à 5 semaines**, avec des points d'étape réguliers avec l'équipe IRM pour ajuster la solution aux réalités opérationnelles et recueillir les retours d'expérience.

## 1.5 Conclusion

Ce premier chapitre a posé les fondations du projet. En partant d'une analyse rigoureuse de l'environnement d'Eneo, nous avons identifié des faiblesses structurelles dans le cycle de vie logiciel et défini une problématique claire, encadrée par des contraintes fortes. Les objectifs sont désormais fixés.

Le chapitre suivant explorera l'arsenal théorique et les pratiques modernes qui nous permettront de concevoir une solution adaptée à ce contexte spécifique.

# Chapitre 2

## Fondements Théoriques et État de l'Art

### 2.1 Introduction

Avant de concevoir une solution technique, il est indispensable de maîtriser les concepts et principes qui la sous-tendent. Le DevSecOps n'est pas simplement une collection d'outils, mais une **culture et une méthodologie** reposant sur des fondements théoriques solides et des pratiques éprouvées. Ce chapitre présente cet état de l'art. Nous définirons les fondements culturels et philosophiques du DevSecOps, puis explorerons les pratiques techniques modernes, avant de conclure sur les métriques permettant d'en mesurer l'efficacité.

### 2.2 Fondements Théoriques du DevSecOps

#### 2.2.1 Du DevOps au DevSecOps : Évolution Nécessaire

Le **DevOps** est né de la volonté de rapprocher les équipes de développement (Dev) et d'exploitation (Ops) pour accélérer les cycles de livraison. Cependant, cette accélération a souvent laissé la sécurité de côté, créant des vulnérabilités critiques.

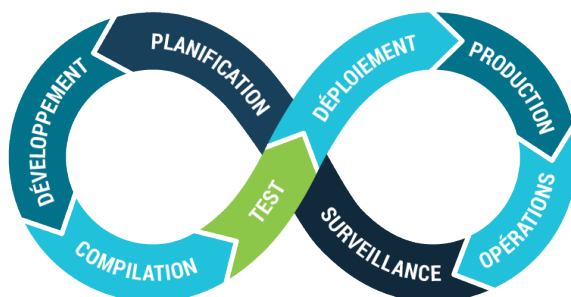


FIGURE 2.1 – Le cycle de vie DevOps sans intégration de la sécurité

Le **DevSecOps** intègre la sécurité (Sec) de manière native dans ce cycle, faisant de celle-ci une responsabilité partagée et automatisée à chaque étape.

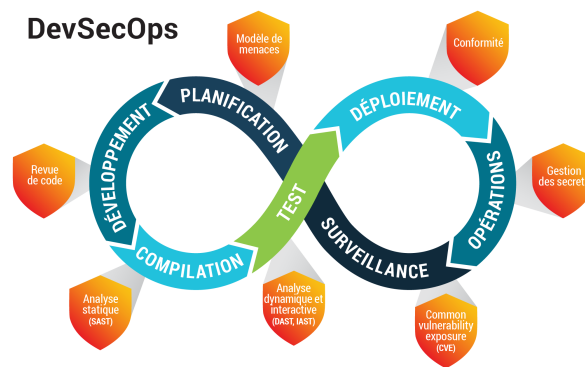


FIGURE 2.2 – Le principe du "Shift Left" : intégrer la sécurité dès la conception

### 2.2.2 Sécurité par Conception (“Secure by Design”)

Le principe de “**Secure by Design**” constitue la pierre angulaire du DevSecOps. Il postule que la sécurité ne doit pas être une surcouche appliquée *a posteriori*, mais une caractéristique intrinsèque du système, pensée dès les premières phases de conception.

Selon l’OWASP Foundation :<sup>1</sup>

« Le Secure-by-Design (SbD) est une approche fondamentale du cycle de vie du développement logiciel qui garantit que la sécurité est intégrée dans les applications et services dès le départ, les rendant résilients face aux menaces et conformes aux politiques réglementaires et organisationnelles. »

Cela implique que les exigences de sécurité soient traitées avec la même importance que les exigences fonctionnelles.

### 2.2.3 Culture de la Collaboration et Responsabilité Partagée

Historiquement, la sécurité était le domaine exclusif d’une équipe dédiée, souvent perçue comme un frein par les développeurs. Le DevSecOps brise ce silo.

Cette responsabilité partagée transforme la sécurité d’une contrainte en un réflexe collectif.

### 2.2.4 Automatisation et Amélioration Continue

L’automatisation est le moteur permettant au DevSecOps de fonctionner à l’échelle et à la vitesse du DevOps. L’objectif : intégrer des contrôles de sécurité automatisés tout au long du cycle de vie. Chaque feedback (résultat de scan, alerte) devient une opportunité d’amélioration selon le cycle vertueux : **Construire** → **Mesurer** → **Apprendre**.

### 2.2.5 Le Principe "Fail Fast, Fail Early"

Ce principe, hérité de la culture Agile, s’applique à la sécurité. Il est exponentiellement moins coûteux de corriger une vulnérabilité lors de l’écriture du code qu’une fois l’application

en production.

## 2.3 Étude de l'Art : Retour d'expérience des acteurs africains

### 2.3.1 L'Impératif DevSecOps et la Transformation des Acteurs Majeurs

La transformation numérique est indissociable d'une accélération de la livraison de services, ce qui impose l'adoption des pratiques DevOps. Pour les entreprises opérant dans des secteurs critiques (finance, télécommunications), l'intégration de la sécurité (DevSecOps) est vitale pour maîtriser les risques réglementaires et opérationnels. L'analyse des cas suivants montre comment des homologues ont géré cette transition.

#### Standard Bank Group (Afrique du Sud)

**Secteur d'Activité :** Services Financiers et Bancaires.

**Problématique initiale :** En 2013, Standard Bank faisait face à un cycle de vie applicatif lent, rigide et coûteux, caractéristique des systèmes hérités (*legacy*). Le délai de mise sur le marché (*Time-to-Market*) atteignait 700 jours pour certains projets majeurs. Les dépendances entre équipes étaient ingérables, et la faible fréquence de déploiement (une à deux fois par an) limitait l'innovation.<sup>31</sup>

**Solution Implémentée :** La banque a opté pour le Scaled Agile Framework (SAFe) pour structurer son agilité à l'échelle. SAFe est une méthode qui aide les grandes entreprises à coordonner de multiples équipes de développement derrière des objectifs partagés. Cela a formalisé et accéléré l'adoption des pratiques DevOps, notamment la création de "trains de lancement agile" (ARTs) et l'automatisation complète de l'environnement de développement et de déploiement (*self-provisioning*) avec des outils comme Chef.<sup>32</sup>

**Résultats et Impact :** Le Time-to-Market a été réduit de 700 jours à 30 jours. La fréquence de déploiement est passée de annuelle/bi-annuelle à mensuelle. La productivité a été accrue de 50% et les coûts réduits de 77%.<sup>31</sup>

#### Safaricom (Kenya)

**Secteur d'Activité :** Télécommunications et Services Financiers Mobiles (M-Pesa).

**Problématique initiale :** Avec l'explosion de l'utilisation de M-Pesa et de ses services numériques, Safaricom a fait face à une augmentation exponentielle des interactions clients et des exigences en matière de sécurité. La difficulté principale était les coûts opérationnels élevés liés au maintien d'une grande équipe de support client et des problèmes de sécurité et de résilience de son infrastructure Cloud (AWS).<sup>33</sup>

**Solution Implémentée :** Safaricom a renforcé son architecture Cloud (AWS) en adoptant une

approche DevSecOps centrée sur la sécurité du Cloud et l'automatisation via l'Intelligence Artificielle (IA). L'Enhanced Cloud Security implique d'intégrer des contrôles de sécurité directement dans l'infrastructure Cloud en automatisant la gestion des accès, le chiffrement et la surveillance, assurant que la sécurité est intégrée par conception et non ajoutée après coup.<sup>33</sup> Un chatbot "Zuri Bot" basé sur l'IA générative a également été déployé pour automatiser le service client.

**Résultats et Impact :** Réduction de 40% du temps de traitement moyen des demandes clients et optimisation des coûts opérationnels du service client de 30%.<sup>33</sup> Mise en place d'une approche de sécurité Cloud proactive.

### Vodacom (Afrique du Sud)

**Secteur d'Activité :** Télécommunications.

**Problématique initiale :** Vodacom était confronté à des risques de sécurité et de conformité liés à la gestion manuelle des accès pour les employés et les contractuels. Cela générait des failles de sécurité potentielles et un risque de non-conformité réglementaire concernant la protection des données clients était élevé.<sup>34</sup>

**Solution Implémentée :** Mise en œuvre d'une solution de gestion des identités et des accès (IAM) de l'éditeur Micro Focus (OpenText). L'IAM est un système centralisé qui gère l'identité numérique de chaque utilisateur et contrôle les droits d'accès, en automatisant la création, la modification et la suppression de ces droits.<sup>34</sup> Le déploiement comprenait l'automatisation du déprovisionnement des droits d'accès logiques et physiques dès le départ d'un collaborateur.<sup>34</sup> L'entreprise a également mis en place une politique de *virtualisation-first* pour moderniser l'infrastructure serveur.<sup>35</sup>

**Résultats et Impact :** Atteinte des objectifs de conformité clés fixés par le Chief Risk Officer. Sécurité accrue des données sensibles grâce à la révocation immédiate et fiable de tous les droits d'accès le jour du départ.<sup>34</sup> Réduction des coûts administratifs.

### MTN South Africa (Afrique du Sud)

**Secteur d'Activité :** Télécommunications.

**Problématique initiale :** Les processus de développement d'applications étaient lents et peu agiles, avec des tests majoritairement manuels.<sup>36</sup> Les systèmes hérités ne permettaient pas de fournir des services à l'échelle et à la vitesse souhaitées.<sup>37</sup>

**Solution Implémentée :** L'approche s'est concentrée sur la plateforme de gestion du cycle de vie des applications (ALM - Application Lifecycle Management) de Micro Focus. L'ALM est une suite d'outils qui gère le produit logiciel de A à Z, assurant la traçabilité des exigences, la gestion des tests et la collaboration entre toutes les équipes impliquées.<sup>36</sup> Le déploiement comprenait le passage des tests fonctionnels, de sécurité et de performance de manuels à entièrement automatisés.

**Résultats et Impact :** Exécution des tests d'applications 75% plus rapide. Réduction du temps de déploiement logiciel de 96%. Économie de 113 heures/homme en un seul mois grâce à l'automatisation.<sup>36</sup>

### 2.3.2 Tableau Comparatif des Stratégies DevSecOps

Le tableau ci-dessous synthétise les approches et en évalue la pertinence pour le contexte spécifique d'Eneo, caractérisé par une forte contrainte de budget (100% open-source).

| Entreprise       | Projet Principal            | Force de la Solution                                                                               | Limite (pour le contexte Eneo)                                                                                                                                              |
|------------------|-----------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Standard Bank    | Transformation SAAFe/DevOps | Transformation à l'échelle, réduction drastique du Time-to-Market.                                 | Dépendance à un Framework coûteux (SAAFe) et à des outils propriétaires, nécessitant un investissement initial massif non compatible avec la contrainte budgétaire d'Eneo.  |
| Safaricom        | Sécurisation Cloud & IA     | Focus sur la sécurité des identités et du chiffrement dans un environnement cloud-native.          | Forte dépendance à des plateformes Cloud (AWS) et à des solutions IA coûteuses. L'approche est centrée sur le Cloud et non sur le Datacenter interne ou l'hybride bas-coût. |
| Vodacom          | IAM et Virtualisation       | Automatisation critique de la gestion des accès et de la conformité.                               | Utilisation de solutions propriétaires (NetIQ Identity Manager) et focus sur la gestion de l'infrastructure plutôt que sur l'analyse de sécurité du code applicatif.        |
| MTN South Africa | ALM et CI/CD                | Automatisation complète des tests (sécurité incluse) et réduction massive du temps de déploiement. | Utilisation d'une suite logicielle coûteuse (Micro Focus ALM) et nécessité d'un changement culturel de grande ampleur.                                                      |

TABLE 2.1 – Tableau comparatif des stratégies DevSecOps des acteurs africains

### 2.3.3 Conclusion du Chapitre

L'existant montre que les leaders africains ont tous adopté des approches DevOps/DevSecOps matures basées sur l'automatisation, mais reposant sur des outils propriétaires coûteux et des infrastructures Cloud. Cette stratégie est incompatible avec les contraintes financières et organisationnelles d'Eneo pour ce POC.

D'où la nécessité de concevoir une solution DevSecOps 100% open-source, adaptée au contexte local, garantissant sécurité, gouvernance centralisée, et automatisation.

# Chapitre 3

## Conception de la Solution et Architecture Technique

### 3.1 Introduction

Ce chapitre constitue le cœur de la démarche d'ingénierie et décrit la phase de conception de la solution DevSecOps. En s'appuyant sur le principe du *Secure by Design* et l'analyse du contexte d'Eneo, j'ai traduit les risques et contraintes identifiés en un cahier des charges précis. Chaque choix technologique est justifié par l'état de l'art, aboutissant à une architecture globale cohérente et robuste.

#### 3.1.1 Héritage de l'État de l'Art et Justification du Design

L'analyse réalisée au Chapitre 2 a mis en évidence que les entreprises africaines les plus avancées dans la transformation numérique (Standard Bank, Safaricom, Vodacom, MTN) ont toutes convergé vers un modèle commun : automatisation à grande échelle, intégration précoce de la sécurité et centralisation de l'observabilité. Malgré leurs différences sectorielles, trois enseignements majeurs se dégagent : (1) les gains les plus importants proviennent d'une automatisation complète du cycle de livraison, (2) la sécurité doit être intégrée très tôt dans la chaîne DevOps (Shift Left), et (3) la visibilité unifiée des risques est indispensable pour réduire les délais de remédiation.

Cependant, l'étude a également montré que ces transformations reposent principalement sur des solutions propriétaires ou des infrastructures Cloud coûteuses (SAFe, AWS, Micro Focus ALM, NetIQ IAM), difficilement transposables dans un contexte à contrainte budgétaire forte comme celui d'Eneo.

La conception présentée dans ce chapitre s'inspire donc des pratiques ayant démontré leur efficacité (automatisation, sécurité embarquée, observabilité continue), tout en les adaptant au contexte réel d'Eneo grâce à un écosystème entièrement open-source. Cette approche permet

d'atteindre la maturité DevSecOps observée chez les leaders, mais avec un coût total de possession quasi nul et une architecture parfaitement maîtrisable en interne.

### 3.2 Démarche de Conception : Le Secure by Design en Pratique

Conformément au principe Secure by Design présenté à la section 2.2.2, notre architecture l'applique à trois niveaux :

- **Infrastructure (IaC)** : Infrastructure définie comme code (`docker-compose.yml`), auditable, reproductible et sécurisée par isolation réseau (`devnet`).
- **Pipeline (CI/CD)** : Sécurité intégrée au `Jenkinsfile` : tout échec d'analyse (SAST, SCA) bloque le build, au même titre qu'une erreur de compilation.
- **Observabilité (Data)** : Rapports de scan normalisés et centralisés, rendant la posture de sécurité mesurable en temps réel (MTTD/MTTR).

### 3.3 Solution Proposée

Pour répondre aux freins observés (Chapitre 1), la solution repose sur trois couches logiques qui garantissent l'exécution séquentielle et la centralisation de la donnée :

1. **La Couche d'Orchestration (Jenkins + Docker)** : Elle est le moteur du pipeline CI/CD. Son rôle est de piloter l'enchaînement des étapes et de garantir la cohérence de l'environnement d'exécution via la conteneurisation.
2. **La Couche de Contrôle (Security Gates - Shift Left)** : Elle intègre les mécanismes de sécurité en amont du déploiement.
  - Ces contrôles sont dits statiques car ils analysent le code et les artefacts sans avoir besoin d'exécuter l'application (contrairement au DAST).
  - Cette couche est composée du SAST (SonarQube), du SCA (Snyk) et du Scan de Conteneur (Trivy) pour couvrir les trois plans du code.
  - **Précision technique** : Ces outils interviennent immédiatement après le `checkout` du code source et l'étape de construction (`mvn clean install`) qui prépare les artefacts binaires et les images Docker à analyser. Ils fournissent un rapport structuré avant même le déploiement.
3. **La Couche d'Observabilité (Elastic Stack)** : Elle est le point d'atterrissage des résultats. Elle est responsable de la normalisation des rapports de sécurité hétérogènes et de la mise à disposition des métriques (MTTR, MTTD) pour le pilotage.

Cette approche modulaire permet de découpler la logique de sécurité des outils spécifiques, garantissant la pérennité et l'évolutivité de la solution.

## 3.4 Spécifications et Cahier des Charges de la Solution

Sur la base de cette architecture logique, nous définissons les exigences techniques du socle initial :

| Risque(s)                 | Exigence                            | Description                                                                                                                                           |
|---------------------------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| O1                        | E1 : Automatisation CI/CD           | La solution doit fournir un pipeline 100% automatisé, du commit au rapport.                                                                           |
| O2, O5                    | E2 : Sécurité « Shift Left »        | La solution doit intégrer des analyses SAST, SCA et de conteneurs à chaque build.                                                                     |
| O3, O4                    | E3 : Observabilité Centralisée      | La solution doit agréger, normaliser et stocker tous les rapports, métriques et logs dans une base de données centralisée et requêtable pour l’audit. |
| O4                        | E4 : Réponse Standardisée           | La solution doit générer des alertes proactives en cas d’incident et fournir des guides de remédiation.                                               |
| Contrainte<br>Deploiement | E5 : Reproductibilité & Portabilité | La solution doit être entièrement conteneurisée pour être portable, reproductible et simuler un environnement multi-services sur une VM unique.       |

TABLE 3.1 – Cahier des charges de la solution

### 3.4.1 Synthèse des Exigences Issues de l’État de l’Art

Les exigences définies pour ce projet proviennent directement des risques observés dans l’environnement d’Eneo (O1 à O5) et des contraintes organisationnelles identifiées lors de l’audit interne. Toutefois, l’analyse de l’état de l’art menée au Chapitre 2 permet de confirmer leur pertinence : les grandes entreprises africaines ayant adopté des approches DevOps/DevSecOps réussies reposent toutes sur des principes d’automatisation, de contrôle continu et de gouvernance centralisée. Ces retours d’expérience valident les choix opérés pour Eneo tout en montrant la nécessité d’une solution complètement open-source, simple à maintenir et adaptée à un environnement sans Cloud public. Ainsi, l’état de l’art n’est pas la source des exigences, mais un appui qui renforce leur justification et leur alignement avec les meilleures pratiques internationales.

## 3.5 Choix Technologiques Pour la Solution

### 3.5.1 Validation et Complétion de la Stack Technologique

L'équipe IRM avait déjà identifié certains composants clés (Jenkins pour CI/CD, Elastic Stack pour l'observabilité) sur la base de retours d'expérience internes et de benchmarks sectoriels.

Il restait juste à :

1. **Valider** ces choix par une analyse comparative formelle
2. **Compléter** la stack avec les outils de sécurité manquants (SAST, SCA, Container Scanning)
3. **Orchestrer** l'intégration cohérente de l'ensemble

Les tableaux suivants présentent les analyses comparatives réalisées, y compris pour les outils pré-sélectionnés.

### 3.5.2 Orchestrateur CI/CD (Exigence E1)

**Besoin :** Automatiser l'intégralité du pipeline, du commit au rapport.

| Outil                                                                                                            | Points Forts                                                                                                                              | Points Faibles                                                                                                 | Verdict       |
|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------|
| <br><b>Jenkins</b><br>Jenkins | Standard de l'industrie, open-source (gratuit), écosystème de plugins immense, flexibilité totale avec le Jenkinsfile (Pipeline as Code). | Interface vieillissante, configuration initiale plus complexe.                                                 | <b>Retenu</b> |
| GitLab CI                                                                                                        | Intégration native à GitLab, syntaxe YAML simple et moderne, runners intégrés.                                                            | La version gratuite a des limites strictes (minutes de calcul, fonctionnalités de sécurité avancées payantes). | <b>Rejeté</b> |
| GitHub Actions                                                                                                   | Similaire à GitLab CI, très puissant, marketplace d'actions riche.                                                                        | Dépendant de l'écosystème GitHub. Non pertinent car le SCM utilisé est GitLab.                                 | <b>Rejeté</b> |

TABLE 3.2 – Comparaison des orchestrateurs CI/CD

### Présentation et Rôle de Jenkins

- **Identification** : Jenkins est le serveur d'automatisation open-source le plus répandu, basé sur Java.
- **Rôle dans la solution** : Jenkins est l'orchestrateur central qui séquence l'exécution de toutes les étapes de CI/CD et de sécurité. Il traduit les exigences du DevSecOps en étapes concrètes et automatisées.

### Architecture Technique et Composants Clés

- **Architecture technique** : Jenkins repose sur une architecture Maître/Agent. Le serveur *Maître* gère l'interface utilisateur, la planification des jobs et l'état des pipelines. Les *Agents* sont des conteneurs légers exécutés par Docker Engine, qui effectuent le travail réel (build, scan, test), garantissant l'isolation des environnements de travail.
- **Fonctionnalités Clés Utilisées (Justification)** :
  - *Pipeline as Code (Jenkinsfile)* : Permet de définir le workflow complet (étapes, outils, conditions) dans un fichier Groovy versionné avec le code, ce qui est crucial pour la traçabilité (Exigence E4).<sup>2</sup>
  - *Plugins* : L'écosystème de plugins (GitLab, SonarQube, Docker, etc.) est indispensable pour interconnecter tous les outils du pipeline de manière standardisée.<sup>3</sup>
  - *Gestion des Credentials* : Jenkins stocke de manière sécurisée les jetons d'accès pour interagir avec SonarQube ou Snyk.<sup>4</sup>

### 3.5.3 Analyse de Sécurité Statique – SAST (Exigence E2)

**Besoin** : Analyser le code source pour détecter les failles (injections, XSS, etc.).

## CHAPITRE 3. CONCEPTION DE LA SOLUTION ET ARCHITECTURE TECHNIQUE

| Outil                                                                                          | Points Forts                                                                                                          | Points Faibles                                                                   | Verdict |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|---------|
| <br>SonarQube | Leader du SAST, analyse multi-langages, règles de qualité du code complètes. Version Community gratuite et puissante. | Nécessite sa propre base de données. Consommation de ressources non négligeable. | Retenu  |
| SonarCloud                                                                                     | Version SaaS de SonarQube, pas d'infrastructure à gérer.                                                              | Payant au-delà d'un certain seuil pour les projets privés.                       | Rejeté  |
| Checkmarx / Veracode                                                                           | Outils commerciaux très puissants, précision élevée.                                                                  | Coût de licence très élevé, complexes, « boîtes noires » propriétaires.          | Rejeté  |

TABLE 3.3 – Comparaison des outils SAST

### Présentation et Rôle de SonarQube

- **Identification** : Plateforme leader d'analyse statique de code (SAST) développée par SonarSource, détectant bugs, vulnérabilités et *code smells*.
- **Rôle dans la solution** : Sert de *Quality Gate* dans le pipeline (Exigence E2 : Shift Left), analysant le code avant le build et bloquant le déploiement si les seuils de sécurité ne sont pas respectés.<sup>5</sup>

### Architecture Technique et Composants Clés

- **Architecture technique** : Architecture trois tiers :
  - *Serveur SonarQube (Java)* : Gère l'UI, les analyses et processus d'arrière-plan.
  - *Base de données PostgreSQL* : Stocke configurations, résultats et métriques historiques.
  - *Scanner* : Outil léger exécuté par Jenkins, analyse le code et transmet les résultats au serveur.<sup>6</sup>
- **Fonctionnalités Clés** :
  - *Quality Gates* : Seuils automatisés (ex : zéro vulnérabilité critique) pour valider ou bloquer le pipeline (répond à E1).
  - *Analyse Multi-langages* : Support PHP/Java/Python pour les applications hétérogènes d'Eneo.

- *Intégration CI/CD* : Retour temps réel sur la sécurité dès le début du cycle de développement.

### 3.5.4 Analyse de Composition Logicielle – SCA (Exigence E2)

**Besoin** : Analyser les dépendances open-source pour détecter les vulnérabilités connues (CVEs).

| Outil                                                                                     | Points Forts                                                                                                          | Points Faibles                                           | Verdict |
|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|---------|
| <br>Snyk | Leader du SCA, base de données de vulnérabilités très à jour, intégration facile via CLI, contextes d'exploitabilité. | Version open source limitée en nombre de tests par mois. | Retenu  |
| OWASP Dependency-Check                                                                    | 100% open-source et gratuit, sans limites. S'intègre bien avec Maven/Jenkins.                                         | Moins rapide que Snyk, taux de faux positifs plus élevé. | Rejeté  |
| GitHub Dependabot                                                                         | Intégration native à GitHub, alertes et PR automatiques.                                                              | Spécifique à l'écosystème GitHub.                        | Rejeté  |

TABLE 3.4 – Comparaison des outils SCA

#### Présentation et Rôle de Snyk CLI

- **Identification** : Outil Cloud spécialisé dans l'Analyse de Composition Logicielle (SCA).<sup>7</sup>
- **Rôle dans la solution** : Scanne via CLI les manifestes de dépendances (`composer.json`) pour identifier les vulnérabilités dans les librairies tierces, répondant à l'Exigence E2.<sup>8</sup>

#### Architecture Technique et Composants Clés

- **Architecture technique** : Modèle Cloud-Backend/Local-Agent. Le CLI s'authentifie via Token sécurisé et transmet les dépendances à la base de connaissances Snyk Cloud pour analyse.<sup>9</sup>
- **Fonctionnalités Clés** :
  - *Authentication API* : Clé API stockée comme secret Jenkins, garantissant accès sécurisé et traçable.
  - *Base de Vulnérabilités* : Combine base propriétaire Snyk et données NVD, offrant précision et mises à jour rapides.<sup>8</sup>

- *Rapports Actionnables* : Recommandations de patch claires avec versions cibles, facilitant la correction.

### 3.5.5 Analyse de Vulnérabilités des Conteneurs (Exigence E2)

**Besoin** : Analyser les images Docker pour détecter les vulnérabilités au niveau des couches système.

| Outil                                                                                      | Points Forts                                                                                  | Points Faibles                                                            | Verdict |
|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|---------|
| <br>Trivy | 100% open-source, rapide, détection exhaustive (OS, dépendances, secrets), facile à intégrer. | Base de données légèrement moins complète que les solutions commerciales. | Retenu  |
| Clair                                                                                      | Open-source, analyse approfondie des couches d'images.                                        | Configuration plus complexe, nécessite une base de données PostgreSQL.    | Rejeté  |
| Anchore Engine                                                                             | Puissant, permet des politiques personnalisées.                                               | Plus lourd, courbe d'apprentissage élevée.                                | Rejeté  |

TABLE 3.5 – Comparaison des outils de scan de conteneurs

#### Présentation et Rôle de Trivy

- **Identification** : Scanner de vulnérabilités open-source (Aqua Security) spécialisé dans l'analyse d'images de conteneurs, systèmes de fichiers et dépendances.<sup>10</sup>
- **Rôle dans la solution** : Scanne les images Docker construites pour identifier les vulnérabilités OS et applicatives, sécurisant la couche infrastructure (Shift Left).<sup>10</sup>

#### Architecture Technique et Composants Clés

- **Architecture technique** : Binaire autonome (Go) maintenant une base de vulnérabilités locale (GitHub Security Advisory), analysant manifestes système et applicatifs des images Docker.
- **Fonctionnalités Clés** :
  - *Scan Multi-couches* : Analyse dépendances applicatives ET packages système (apt, yum, apk) pour une couverture complète.
  - *Sortie JSON structurée* : Facilite l'intégration au pipeline de normalisation.
  - *Détection de secrets* : Identifie credentials accidentellement inclus dans les images.

### 3.5.6 Plateforme d'Observabilité (Exigence E3 & E4)

**Besoin :** Centraliser, analyser, visualiser tous les logs et métriques, et déclencher des alertes.

| Outil                                                                                              | Points Forts                                                                                                                | Points Faibles                                                             | Verdict |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|---------|
| <br>Elastic Stack | Plateforme tout-en-un (Logs, Métriques, APM, Sécurité). Licence Basic gratuite avec toutes les fonctionnalités nécessaires. | Consommation de ressources élevée. Complexité de la configuration avancée. | Retenu  |
| Splunk (Free)                                                                                      | Moteur de recherche de logs très puissant.                                                                                  | Version gratuite limitée à 500 Mo/jour d'ingestion.                        | Rejeté  |
| Graylog (Open)                                                                                     | Alternative open-source populaire.                                                                                          | Moins complet que la stack Elastic (pas de brique APM intégrée).           | Rejeté  |

TABLE 3.6 – Comparaison des plateformes d'observabilité

#### Présentation et Rôle de l'Elastic Stack

L'Elastic Stack (**ELK** : Elasticsearch, Logstash, Kibana) est une suite open-source de collecte, analyse et visualisation de données en temps réel, pilier de l'*Observabilité Centralisée* (Exigence E4).<sup>11</sup>

- **But :** Collecter les trois piliers de l'observabilité (*Logs, Métriques, Traces*) pour une vue unifiée de l'état de santé et de sécurité du pipeline DevSecOps et de l'application scannée.
- **Rôle dans la solution :** Permet le *Threat Modeling* en corrélant événements de sécurité du pipeline (échecs SonarQube/Trivy) avec le comportement applicatif en test/production. Sert de moteur d'*Alerting* pour la réponse standardisée (Exigence E5).

#### Architecture Composante et Fonctionnalités Clés

Architecture modulaire utilisant cinq composants : serveur, base de données, visualiseur et deux agents de collecte.

#### Composants de Stockage et d'Analyse

##### 1. Elasticsearch

## CHAPITRE 3. CONCEPTION DE LA SOLUTION ET ARCHITECTURE TECHNIQUE

| Catégorie                                                                                                      | Identification & Rôle                                                          | Rôle dans le Projet                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <br>Moteur de Base de Données | Moteur de recherche distribué basé sur Apache Lucene.                          | <b>Stockage Centralisé (E4)</b> : Référentiel unique pour logs, métriques, traces et événements de sécurité.                                                 |
| Architecture                                                                                                   | Modèle NoSQL orienté document avec indexation et requêtes plein texte rapides. | <b>Indexation Sécurité</b> : Index séparés ( <code>filebeat-pipeline-*</code> , <code>apm-*</code> ) pour corrélation rapide lors d'incidents. <sup>12</sup> |

### 2. Kibana

| Catégorie                                                                                                        | Identification & Rôle                                                     | Rôle dans le Projet                                                                                                           |
|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <br>Interface de Visualisation | UI web pour interroger, analyser et visualiser les données Elasticsearch. | <b>Dashboards Sécurité (E4)</b> : Tableaux de bord DevSecOps (temps de build, échecs de scan, dette technique). <sup>13</sup> |
| Architecture                                                                                                     | Fonctionne au-dessus d'Elasticsearch, sans stockage propre.               | <b>Système d'Alerting (E5)</b> : Watchers intégrés (licence Basic) pour notifications proactives (email, Slack).              |

### Composants de Collecte

Les *Beats* (agents légers spécialisés) collectent les données en respectant le principe du moindre privilège.

#### 3. Filebeat

## CHAPITRE 3. CONCEPTION DE LA SOLUTION ET ARCHITECTURE TECHNIQUE

| Catégorie                                                                                               | Identification & Rôle                                                              | Rôle dans le Projet                                                                                                  |
|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <br>Collecteur de Logs | Shipper léger pour transférer logs en temps réel vers Elasticsearch. <sup>14</sup> | <b>Centralisation Logs Pipeline</b> : Déployé dans les conteneurs (Jenkins, SonarQube) pour collecter logs du CI/CD. |
| Architecture                                                                                            | Basé sur Go, faible consommation de ressources.                                    | <b>Normalisation Logs</b> : Modules (Docker) structurent et normalisent automatiquement les logs conteneurs.         |

### 4. Metricbeat

| Catégorie                                                                                                    | Identification & Rôle                                                                    | Rôle dans le Projet                                                                       |
|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <br>Collecteur de Métriques | Shipper collectant métriques système et infrastructure (CPU, RAM, réseau). <sup>15</sup> | <b>Surveillance Infrastructure</b> : Collecte métriques VM hôte et conteneurs Docker.     |
| Architecture                                                                                                 | Modules spécifiques interrogeant API Docker et OS.                                       | <b>Détection Anomalies</b> : Identification dérives performance ou activité malveillante. |

### 5. APM-Server

| Catégorie                                                                                          | Identification & Rôle                                                             | Rôle dans le Projet                                                                                                                   |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <br>Serveur APM | Point d'entrée pour réception et traitement des traces et métriques applicatives. | <b>Traces Applicatives (E4)</b> : Reçoit données de l'agent PHP/Laravel instrumenté dans Performa, vue bout-en-bout des performances. |
| Architecture                                                                                       | Serveur intermédiaire gérant l'ingestion avant envoi à Elasticsearch.             | <b>Analyse Transactions</b> : Identifie goulots d'étranglement (BDD lente, API externe, fonctions coûteuses).                         |

### 3.5.7 Technologie de Conteneurisation (Exigence E5)

**Besoin** : Isoler les services, garantir la reproductibilité et simuler un environnement multi-services sur une VM unique.

## CHAPITRE 3. CONCEPTION DE LA SOLUTION ET ARCHITECTURE TECHNIQUE

| Technologie                                                                                                  | Points Forts                                                                                          | Points Faibles                                                      | Verdict |
|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|---------|
| <br>Docker & Docker Compose | Standard industrie, communauté vaste. Docker Compose : infrastructure multi-conteneurs en YAML (IaC). | Pas d'orchestration production à grande échelle (pas de HA native). | Retenu  |
| Kubernetes (K8s)                                                                                             | Leader orchestration : scaling, auto-réparation, haute disponibilité.                                 | Courbe d'apprentissage élevée. Lourd pour PoC sur VM unique.        | Rejeté  |
| Podman                                                                                                       | Alternative sans démon, plus sécurisé.                                                                | Moins mature, écosystème restreint. podman-compose peu répandu.     | Rejeté  |

TABLE 3.7 – Comparaison des technologies de conteneurisation

### Présentation et Rôle de Docker et Docker Compose

- **Identification** : Docker : technologie de conteneurisation fournissant images portables et isolées. Docker Compose : orchestration légère pour développement et PoC.<sup>16</sup>
- **Rôle dans la solution** : Fondation technique rendant la solution portable, reproductible et gérable sur VM unique (Exigence E6). Chaque service (Jenkins, GitLab, Elastic Stack, Application) est un conteneur isolé.

### Architecture Technique et Composants Clés

- **Architecture technique** : Docker Engine (*daemon*) gère images et conteneurs. Docker Compose utilise un fichier YAML déclaratif définissant la stack multi-conteneurs.<sup>17</sup>
- **Fonctionnalités Clés** :
  - *Infrastructure as Code (IaC)* : `docker-compose.yml` garantit démarrage identique de l'environnement simulé partout.
  - *Réseaux Virtuels* : Communication inter-conteneurs par noms de service (`http://sonarqube:9000`), indépendante des IP hôtes (portabilité).
  - *Isolation des Services* : Résolution conflits de dépendances (versions Java, Python) et renforcement sécurité.

### 3.5.8 Technologie de Gestion de Code Source (SCM)

**Exigence(s) liée(s)** : E1 (Déclenchement CI/CD).

## CHAPITRE 3. CONCEPTION DE LA SOLUTION ET ARCHITECTURE TECHNIQUE

| Outil                                                                                                       | Points Forts                                                           | Points Faibles                                                             | Verdict |
|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------|---------|
| <br>GitLab<br>Auto-Hébergé | Flexibilité totale, contrôle des données. Version gratuite disponible. | Coût maintenance élevé (VM, administration).                               | Retenu  |
| GitHub                                                                                                      | Très populaire, Marketplace d'Actions riche, Webhooks robustes.        | SCM non standardisé à l'interne. Données hébergées en externe.             | Rejeté  |
| Bitbucket                                                                                                   | Intégration native avec suite Atlassian (Jira).                        | Version gratuite limitée. Moins flexible que GitLab pour auto-hébergement. | Rejeté  |

TABLE 3.8 – Comparaison des systèmes de gestion de code source

### Présentation et Rôle de GitLab (Auto-Hébergé)

- **Identification** : Plateforme complète pour le cycle DevSecOps, utilisée principalement comme SCM.<sup>18</sup>
- **Rôle dans la solution** : Source de vérité pour le code applicatif et point de déclenchement du pipeline. Auto-hébergement stratégique pour maîtrise totale de la plateforme.

### Architecture Technique et Composants Clés

- **Architecture technique** : Application monolithique conteneurisable intégrant serveur Web, base PostgreSQL et dépôt de fichiers (Gitaly).
- **Fonctionnalités Clés** :
  - *Webhooks* : Mécanisme satisfaisant l'Exigence E1. Événements de code (*push* branche principale) signalés immédiatement à Jenkins pour initier le pipeline.<sup>19</sup>
  - *Gestion des Secrets* : Stockage sécurisé des identifiants (Tokens) récupérés par Jenkins via mécanismes de *secrets management*.<sup>4</sup>

## 3.6 Architecture Globale Cible

L'architecture cible implémente le principe *Security as Code* en intégrant les outils de sécurité directement dans le CI/CD. Basée sur conteneurs Docker et outils Open-Source (GitLab, Jenkins, Elastic Stack), elle garantit reproductibilité, évolutivité et traçabilité.

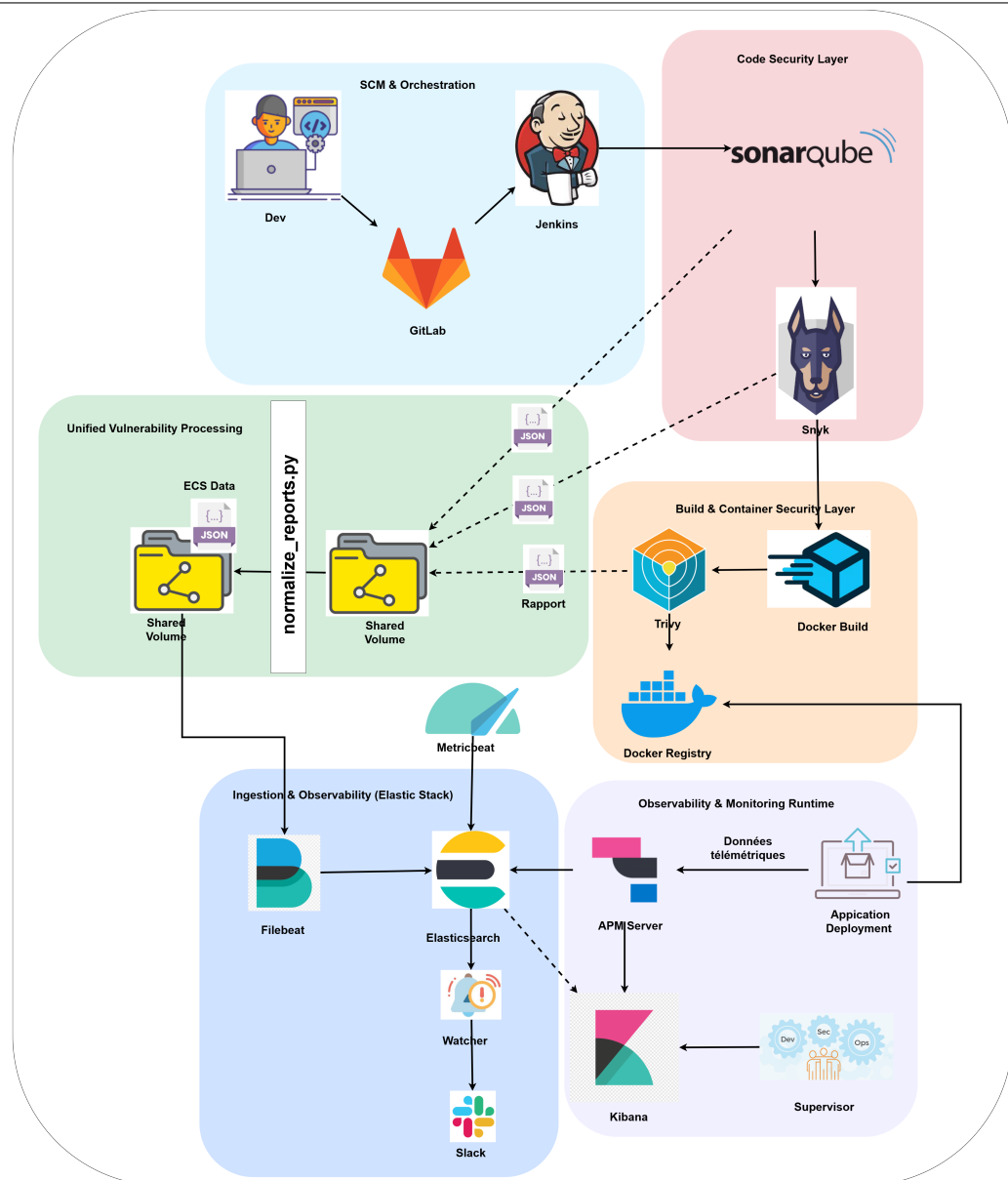


FIGURE 3.1 – Architecture Globale DevSecOps Cible

### 3.6.1 Le Flux d'Orchestration et les Scans de Sécurité

Le pipeline est déclenché par un `git push` vers GitLab, qui envoie un Webhook à Jenkins. Jenkins orchestre l'exécution du `Jenkinsfile`, intégrant les outils de sécurité :

- **SonarQube** : SAST et évaluation qualité code (Bugs, Code Smells, Dette Technique).
- **Snyk** : SCA des dépendances, focus sur vulnérabilités exploitables.
- **Trivy** : Scan vulnérabilités des images Docker (Sécurité Conteneurs).

### 3.6.2 L'Unification des Données et l'Observabilité

Le système repose sur la normalisation et l'unification des rapports :

- Chaque outil génère un rapport JSON transformé par `normalize-reports.py` en format standardisé pour agrégation.
- Rapports normalisés écrits dans volume partagé (`REPORTS_DIR`), ingérés par Filebeat vers Elasticsearch.
- Trois Pipelines d’Ingestion Elasticsearch (SonarQube, Snyk, Trivy) enrichissent les documents et calculent scores de risque.

### 3.6.3 Le Système de Réponse et de Mesure (MTTD/MTTR)

L’Elastic Stack fonctionne comme système de réponse active :

- Kibana héberge six Dashboards pour vision unifiée : sécurité, performance, métriques d’efficacité (MTTD/MTTR).
- Elasticsearch Watcher automatise les alertes critiques :
  - Alerte immédiate vulnérabilités critiques et exploitables vers Slack.
  - Mise à jour automatique statut vulnérabilités pour calcul MTTR.

### 3.6.4 Valeur Ajoutée et Spécificité de la Solution Proposée

La solution conçue pour Eneo se distingue des approches propriétaires observées chez les acteurs africains étudiés (Standard Bank, Safaricom, Vodacom, MTN) en apportant une intégration plus unifiée, plus lisible et entièrement compatible avec les contraintes budgétaires locales. Elle combine automatisation, sécurité et observabilité dans un écosystème 100% open-source, garantissant indépendance et maîtrise totale des coûts.

- **Observabilité unifiée orientée risque** : un tableau de bord unique consolidant SAST, SCA, container scanning, permettant un pilotage clair par la Direction des Risques.
- **Suivi automatique du MTTR et détection des corrections** : mesure dynamique de la performance de remédiation, fonctionnalité absente des solutions concurrentes basées sur outils propriétaires.
- **Indépendance technologique et TCO minimal** : architecture entièrement open-source, sans aucun coût de licence, contrairement aux solutions SAFe, AWS, ALM ou IAM utilisées par les leaders du marché.
- **Chaîne DevSecOps cohérente et intégrée** : sécurité “Shift Left” homogène grâce à un pipeline unique Jenkins → Snyk/Trivy/SonarQube → Elastic Stack, assurant une vision continue et fiable du risque.

En synthèse, cette architecture offre à Eneo un niveau de visibilité, de réactivité et d’autonomie équivalent aux grandes organisations, mais avec une simplicité opérationnelle et un coût d’adoption incomparablement plus faible.

### 3.7 Conclusion

Ce chapitre a défini une architecture DevSecOps complète justifiant chaque choix technologique selon les exigences du chapitre 1. Les outils open-source (Jenkins, SonarQube, Snyk, Trivy, Elastic Stack) et Docker Compose garantissent faisabilité technique, reproductibilité et respect de la contrainte budgétaire zéro-coût d'Eneo.

L'architecture répond aux six exigences :

- **E1 (Automatisation CI/CD)** : Jenkins et `Jenkinsfile`.
- **E2 (Sécurité Shift Left)** : Intégration SonarQube, Snyk, Trivy dans le pipeline.
- **E3 (Observabilité Centralisée)** : Elastic Stack et normalisation.
- **E4 (Réponse Standardisée)** : Watchers Elasticsearch et canaux d'alerting.
- **E5 (Reproductibilité)** : Docker Compose et Infrastructure as Code.

Le chapitre suivant présentera l'implémentation concrète : configurations pour rendre opérationnels le pipeline CI/CD et l'infrastructure conteneurisée. Nous analyserons le fichier `docker-compose.yml`, la construction de l'agent Jenkins personnalisé, et la configuration de l'intégration Git Jenkins-GitLab.

## Chapitre 4

# Mise en Œuvre de l'Infrastructure de Base et Stabilisation des Flux

### 4.1 Introduction

Après avoir posé les fondations conceptuelles, il s'agit désormais de bâtir un pipeline opérationnel capable d'orchestrer les analyses de sécurité de manière fiable et automatisée. Ce chapitre décrit la mise en place de l'infrastructure technique et la validation du premier parcours complet des données. Cette étape est déterminante : elle établit l'ossature sur laquelle reposeront toutes les fonctionnalités avancées (observabilité, alerting, indicateurs).

Les objectifs sont clairs :

- **Déployer une infrastructure conteneurisée cohérente** : Mettre en place, via Docker Compose, l'ensemble des services essentiels (GitLab, Jenkins, SonarQube, Trivy, Elastic Stack) conformément à l'Exigence E5.
- **Garantir un flux de données maîtrisé** : Vérifier qu'un simple push déclenche automatiquement les analyses de sécurité et que les rapports bruts sont correctement produits et stockés dans le volume partagé.

À l'issue de cette étape, le pipeline fonctionne de bout en bout et assure l'exécution automatique et intégrée des analyses de sécurité.

### 4.2 Le Cœur de l'Infrastructure : Docker Compose

L'infrastructure du projet repose sur Docker Compose, un outil d'orchestration léger permettant de définir et de gérer une application multi-conteneurs via un fichier YAML déclaratif. Ce choix, justifié au chapitre 3, répond aux contraintes de reproductibilité (Exigence E5) et de déploiement sur une VM unique.<sup>16</sup>

### 4.2.1 Présentation du fichier d'orchestration

Le fichier `docker-compose.yml` décrit l'intégralité de l'écosystème DevSecOps. Il définit 13 services répartis en trois catégories fonctionnelles :

| Catégorie     | Services                                                | Rôle                                                  |
|---------------|---------------------------------------------------------|-------------------------------------------------------|
| SCM & CI/CD   | GitLab, Jenkins                                         | Gestion du code source et orchestration du pipeline   |
| Sécurité      | SonarQube, PostgreSQL (DB SonarQube), Trivy             | Analyse SAST, SCA et conteneurs                       |
| Observabilité | Elasticsearch, Kibana, Filebeat, Metricbeat, APM Server | Centralisation, visualisation, alerting et monitoring |

TABLE 4.1 – Répartition des services Docker Compose

### 4.2.2 Configuration des Services Clés

#### GitLab : Source de Vérité du Code

Le service GitLab utilise l'image officielle `gitlab-ce` et expose les ports nécessaires pour l'interface web, le HTTPS et le SSH. (**Voir configuration détaillée en Annexe A.1.1**). Pour renforcer la sécurité préventive, les services ont été configurés pour être accessibles uniquement via HTTPS.

Points techniques importants :

- **Hostname** : Le nom `gitlab.kewel.local` est résolu localement via le fichier `/etc/hosts` de la machine hôte, créant une expérience proche d'un environnement de production.
- **Volumes persistants** : Les données de configuration, logs et dépôts Git sont stockés dans des volumes locaux, garantissant la persistance des données même après un redémarrage des conteneurs.
- **Extra Hosts** : Permet à GitLab de résoudre le nom `jenkins.kewel.local` pour envoyer les Webhooks.<sup>19</sup>

#### Jenkins : Orchestrateur Central du Pipeline

Jenkins est déployé via une image personnalisée construite à partir d'un `Dockerfile` (détaillé plus loin). Cette personnalisation est essentielle pour embarquer tous les outils de sécurité nécessaires au pipeline. (**Voir configuration détaillée en Annexe A.1.2**).

Éléments architecturaux critiques<sup>20</sup> :

- **Volume /shared** : Point de montage bidirectionnel entre Jenkins et Filebeat. C'est dans ce répertoire que seront écrits tous les rapports de sécurité (bruts et normalisés).

- **Socket Docker** : Le montage du socket Docker permet à Jenkins (agent) d'utiliser le Docker Engine de l'hôte pour construire les images et exécuter Trivy.
- **Optimisations JVM** : Les paramètres `JAVA_OPTS` limitent la consommation mémoire de Jenkins, essentiel pour un déploiement sur une VM aux ressources limitées.

### SonarQube et PostgreSQL : Analyse de Qualité

SonarQube nécessite une base de données PostgreSQL pour stocker les résultats d'analyse.<sup>5</sup> Cette dépendance est gérée via l'attribut de dépendance Docker. (**Voir configuration détaillée en Annexe A.1.3**).

### Elastic Stack : Centralisation et Observabilité

L'Elastic Stack est déployée en mode *single-node* pour simplifier le PoC, avec authentification activée.<sup>11</sup> (**Voir configuration détaillée en Annexe A.1.4**).

Particularités :

- **Mode single-node** : Désactive la découverte de clusters, adapté pour un PoC sur une seule machine.
- **Sécurité** : Authentification par mot de passe pour protéger l'accès aux données sensibles.

### Filebeat et Metricbeat : Collecteurs de Données

Ces agents légers sont configurés pour collecter les logs et métriques et les envoyer vers Elasticsearch. (**Voir configuration détaillée en Annexe A.1.5**).

**Point de sécurité** : Le volume `/shared` est monté en *lecture seule* pour Filebeat, respectant le principe du moindre privilège.<sup>14</sup>

### 4.2.3 Le Réseau Docker `devnet`

Tous les services sont connectés à un réseau bridge externe nommé `devnet`. Ce réseau doit être créé manuellement avant le premier démarrage (**voir commande en Annexe A.3.1**). Avantages du réseau personnalisé<sup>17</sup> :

- **Résolution DNS automatique** : Les conteneurs peuvent se référencer par leur nom de service.
- **Isolation** : Les conteneurs peuvent communiquer entre eux mais sont isolés du réseau de l'hôte, renforçant la sécurité.

### 4.2.4 Gestion de la Persistance des Données

Le fichier d'infrastructure définit plusieurs volumes Docker nommés pour garantir la persistance des données critiques, comme détaillé dans le tableau ci-dessous.

TABLE 4.2 – Volumes persistants de l'infrastructure

| Volume         | Données Stockées                                        |
|----------------|---------------------------------------------------------|
| gitlab-data    | Dépôts Git, utilisateurs, projets GitLab                |
| jenkins_home   | Configuration Jenkins, jobs, plugins, historique builds |
| sonardb_data   | Base de données PostgreSQL (analyses SonarQube)         |
| sonarqube_data | Données critiques SonarQube (config, plugins)           |
| es_data        | Index Elasticsearch (rapports de sécurité, métriques)   |
| trivy-cache    | Base de données de vulnérabilités Trivy                 |

TABLE 4.3 – Volumes persistants de l'infrastructure

### 4.3 Préparation de l'Agent Jenkins Personnalisé

L'orchestration du pipeline de sécurité nécessite que l'agent Jenkins dispose de tous les outils d'analyse embarqués. Plutôt que d'installer ces outils manuellement, nous construisons une image Docker personnalisée. (**Le Dockerfile complet est disponible en Annexe A.2.1**).

#### 4.3.1 Justification des Choix Techniques

##### Accès au Docker Engine de l'Hôte

L'installation de Docker CLI dans le conteneur Jenkins et le montage du socket permettent à Jenkins de piloter le Docker Engine de l'hôte. Cette architecture *Docker-in-Docker* (DinD) est nécessaire pour construire les images de l'application et exécuter Trivy. Un vrai conteneur DinD aurait nécessité des privilèges élevés, créant un risque de sécurité ; le montage du socket est un compromis acceptable pour un PoC.

##### Outils de Build Multi-Langages

L'application cible (Babyfoot) utilise deux technologies : Backend (Spring Boot/Java) et Frontend (React/JavaScript). L'agent Jenkins doit donc supporter les deux chaînes de compilation, d'où l'installation de Maven et Node.js dans l'image personnalisée.

##### Snyk CLI et Authentification

Snyk CLI est installé globalement via npm. L'authentification se fait via un token stocké dans les *Credentials* Jenkins, injecté au runtime dans le pipeline (**voir Annexe A.4.4**).

### Script de Normalisation Embarqué

Le script Python de normalisation est copié directement dans l'image. Cette approche garantit que le script est toujours disponible et versionné avec l'infrastructure, quelle que soit la branche Git en cours de build.

## 4.4 Configuration de l'Intégration GitLab-Jenkins

L'automatisation du pipeline repose sur la communication bidirectionnelle entre GitLab (SCM) et Jenkins (CI/CD).

### 4.4.1 Génération et Échange de Clés SSH

Jenkins doit pouvoir cloner les dépôts GitLab via SSH. Cette opération nécessite l'échange de clés publiques/privées. La génération s'effectue dans le conteneur Jenkins (**voir Annexe A.3.2**) et la clé publique est ensuite ajoutée dans l'interface de GitLab. La validation de la connexion peut être testée via SSH (**voir Annexe A.3.3**).

### 4.4.2 Configuration des Webhooks GitLab

Les Webhooks permettent à GitLab de notifier Jenkins immédiatement après un `git push`, déclenchant ainsi le pipeline automatiquement. La configuration s'effectue dans les paramètres du projet GitLab en pointant vers l'URL du projet Jenkins. La bonne réception des événements peut être confirmée en consultant les logs de Jenkins (**voir Annexe A.3.1**).

### 4.4.3 Gestion des Credentials Jenkins

Jenkins doit stocker de manière sécurisée plusieurs types de credentials pour interagir avec les outils externes. Ils sont gérés via le gestionnaire de credentials de Jenkins et jamais exposés en clair.

| ID          | Type        | Valeur             | Usage                     |
|-------------|-------------|--------------------|---------------------------|
| snyk-api    | Secret text | Token Snyk         | Authentification Snyk CLI |
| sonar-token | Secret text | Token Sonar-Qube   | Analyse SonarQube         |
| gitlab-ssh  | SSH Key     | Clé privée Jenkins | Clone des dépôts GitLab   |
| docker-reg  | User/Pass   | Docker Hub login   | Push des images Docker    |

TABLE 4.4 – Credentials nécessaires pour le pipeline

### 4.5 Validation du Flux Initial : Premier Pipeline

Avec l'infrastructure déployée, nous validons le flux de données initial en exécutant un premier build complet.

#### 4.5.1 Structure du Pipeline

Le pipeline est défini de manière déclarative et comprend 7 étapes séquentielles (**voir structure complète en Annexe A.4.1**). Chaque étape a une responsabilité claire et capture des métadonnées temporelles.

#### 4.5.2 Capture des Timestamps et Métadonnées Git

Le premier stage est crucial car il définit le point de référence temporel T0 (Code Introduction Time) utilisé pour le calcul du MTTD-CI. Les métadonnées (Commit, Branche, Auteur) sont extraites via des commandes Git et stockées dans des variables d'environnement (**voir Annexe A.4.2**).

#### 4.5.3 Exécution des Analyses de Sécurité

Chaque outil de sécurité (SonarQube, Snyk, Trivy) est exécuté dans un stage dédié. Par exemple, pour le Backend, SonarQube est déclenché via Maven. Le script capture le timestamp de fin d'analyse (T1), récupère les rapports bruts via l'API SonarQube et génère un fichier de métadonnées (**voir code du stage en Annexe A.4.3**).

#### 4.5.4 Architecture du Répertoire Partagé

Le volume partagé `/shared` suit une structure hiérarchique stricte pour faciliter le traitement par Filebeat (**voir arborescence en Annexe ??**). Cette structure isole chaque build et organise les rapports par outil, permettant l'utilisation de motifs simples pour la collecte des logs.

#### 4.5.5 Génération des Rapports Bruts

À l'issue de chaque analyse, les rapports bruts sont écrits dans le volume partagé. Ces rapports sont au format JSON natif de chaque outil (Snyk, SonarQube, Trivy), non encore normalisés, mais contiennent toutes les données nécessaires pour les étapes suivantes.

| Fichier                | Outil     | Contenu                                        |
|------------------------|-----------|------------------------------------------------|
| sonarqube-backend.json | SonarQube | Issues (VULNERABILITY, BUG), métriques qualité |
| snyk-backend.json      | Snyk      | Vulnérabilités dépendances, CVE                |
| trivy-backend.json     | Trivy     | Vulnérabilités image Docker, packages OS       |

TABLE 4.5 – Exemple de rapports bruts générés

### 4.6 Résolution des Noms de Domaine Locaux

Pour simuler un environnement de production avec des noms de domaine personnalisés (`gitlab.kewel.local`, etc.), la résolution DNS locale est configurée sur la machine hôte via le fichier `hosts` (voir **configuration en Annexe A.3.4**). Cela permet aux services de communiquer via des URL stables.

### 4.7 Conclusion

Ce chapitre a décrit en détail la mise en œuvre de l'infrastructure DevSecOps et la validation du flux de données initial. Les réalisations de cette première phase sont les suivantes :

- **Infrastructure conteneurisée opérationnelle** : 10 services déployés via Docker Compose, communiquant sur le réseau `devnet`.
- **Agent Jenkins personnalisé** : Image Docker embarquant tous les outils de build et de sécurité nécessaires.
- **Intégration GitLab-Jenkins sécurisée** : Échange de clés SSH et configuration des Webhooks fonctionnelle.
- **Pipeline CI/CD fonctionnel** : Exécution réussie des 7 stages avec capture des timestamps T0 et T1.
- **Génération des rapports bruts** : Production réussie des rapports JSON structurés dans le volume partagé.

L'infrastructure est désormais **stable** et le flux de données est **validé**. Cependant, les rapports générés sont encore au format natif de chaque outil. Le chapitre suivant détaillera la transformation de ces rapports bruts en un format unifié (Elastic Common Schema) et leur ingestion dans Elasticsearch.

# Chapitre 5

## Mise en Œuvre de l'Observabilité

### 5.1 Introduction : Au-delà des Logs de Sécurité

L'exigence **E3** du cahier des charges imposait une "Observabilité Centralisée". Dans une approche DevSecOps moderne, la sécurité ne peut être isolée de la performance opérationnelle. Un serveur surchargé peut masquer une attaque par déni de service (DDoS), et une application lente peut décourager les utilisateurs autant qu'une faille de sécurité.<sup>38</sup>

Pour répondre à ce besoin, nous avons déployé une architecture d'observabilité complète reposant sur les **Trois Piliers** :

1. **Les Logs** : Pour l'audit de sécurité et les événements discrets (traités via la normalisation et Filebeat).
2. **Les Métriques** : Pour surveiller la santé des ressources (CPU, RAM) via Metricbeat.
3. **Les Traces (APM)** : Pour suivre le cheminement des transactions utilisateurs de bout en bout.

### 5.2 Pilier 1 : Normalisation et Collecte des Logs de Sécurité

Dans un écosystème CI/CD moderne, intégrer des outils de sécurité (SAST, SCA) est indispensable — mais cela introduit un obstacle majeur : **l'hétérogénéité des données**. Snyk, Trivy et SonarQube produisent chacun des JSON propriétaires, impossibles à corréler ou visualiser ensemble, rendant tout calcul de métriques (MTTR, MTTD) inexploitable en l'état.

La solution repose sur une chaîne de traitement en **cinq étapes**, convertissant ces rapports disparates en une source unifiée structurée en documents Parent/Enfant.

#### — Le Schéma de Normalisation (ECS)

Pour éliminer l'incohérence des formats, nous adoptons l'**Elastic Common Schema (ECS)**.<sup>21</sup> Ce standard open-source impose une taxonomie commune pour logs, métriques et traces. En normalisant Snyk, Trivy et SonarQube sous ECS, chaque information (comme

la sévérité, stockée dans `vulnerability.severity`) devient homogène, immédiatement corrélable et exploitable dans Elasticsearch.

### — Architecture Logique Globale

Le diagramme suivant illustre le flux complet, de la génération des rapports bruts jusqu'à la visualisation dans Kibana. Ce schéma servira de guide pour les points aborder plus bas.

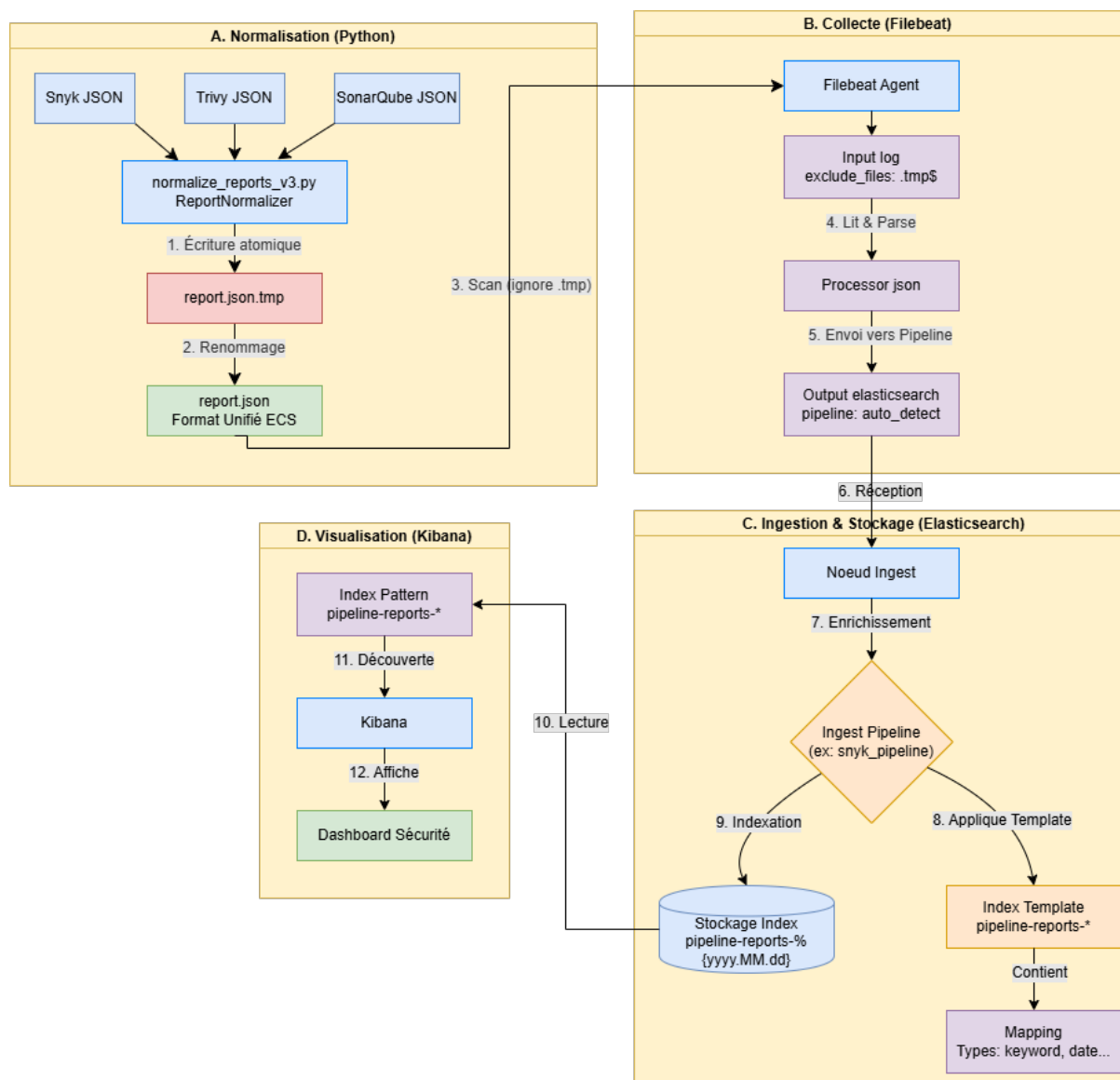


FIGURE 5.1 – Flux de données de la génération des rapports bruts à l'ingestion Elasticsearch

### 5.2.1 Normalisation et Splitting (Le Cœur)

Le défi majeur résidait dans les formats JSON propriétaires de chaque outil. La solution implémentée repose sur deux scripts Python exécutés par Jenkins :

Le script Python<sup>22</sup> (illustré dans le Bloc A du schéma) est le cœur de la transformation. Il gère la logique métier la plus lourde en pré-traitement.

### Architecture du script (Classe `ReportNormalizer`)

Le script est encapsulé dans une classe `ReportNormalizer`. Cette approche orientée objet permet de gérer la complexité en isolant la logique de normalisation spécifique à chaque outil dans des méthodes dédiées.

### Transformation par outil

La classe contient des méthodes privées pour chaque format source :

- `_normalize_snyk()` : Gère les rapports Snyk (SCA et Snyk Code). Elle extrait les vulnérabilités (`vulnerabilities`), les identifiants CVE, les informations d'exploitabilité (`exploitMaturity`), et les informations de package.
- `_normalize_trivy()` : Gère les rapports Trivy (scan de conteneurs). Elle normalise les résultats (`Results`), en se concentrant sur les packages système et les vulnérabilités de l'OS.
- `_normalize_sonarqube()` : Gère les rapports SonarQube (qualité de code). Elle transforme les issues (Bugs, Vulnérabilités, Code Smells) en un format compatible et extrait les métriques de qualité.

### Le bloc `unified` : Clé de l'unification

L'innovation majeure du script est la création du bloc `unified`. Chaque vulnérabilité, quel que soit l'outil, est mappée vers cette structure standardisée, basée sur l'Elastic Common Schema (ECS).<sup>21</sup>

**Structure du bloc `unified` :**

- `vulnerability_id` : L'identifiant unique (CVE, Snyk ID, Sonar Rule).
- `severity` : Sévérité normalisée (ex : `critical`, `high`).
- `status` : Statut (toujours `open` à ce stade).
- `assignee` : Développeur responsable (si disponible).
- `location` : Fichier ou ressource impactée.

Ce bloc est ce qui permet la corrélation *cross-tool* et la création de dashboards unifiés dans Kibana.<sup>13</sup>

### Calcul du MTTD-CI (*Mean Time To Detect - CI*)

Le script calcule une métrique de détection précoce, le "Mean Time To Detect - CI".

- **Formule** :  $(T1 - T0)/N$
- Où **T0** est le début du build, **T1** la fin du scan, et **N** le nombre de vulnérabilités trouvées.
- **Stockage** : Le résultat est stocké dans le champ `metrics.mtttd.build_data.mtttd_hours` (renommé en `mtttd.current_build_hours` par l'Ingest Pipeline).

### Initialisation de la structure MTTR

Le script prépare le terrain pour le Watcher 1 (Calculateur MTTR). Il s'assure que les champs requis par le Watcher (`LastSeen`, `resolution_date`) sont présents, en initialisant `resolution_date` à `1970-01-01T00:00:00Z` (valeur par défaut pour "ouvert") pour garantir la conformité du schéma lors de l'ingestion.

### Écriture atomique

Le rapport normalisé est écrit dans un fichier temporaire (`report.json.tmp`). Une fois l'écriture terminée, le fichier est renommé (`report.json`) en utilisant `os.rename`.<sup>22</sup>

### Splitting (Le Découpage) – `split_reports.py`

Le rapport normalisé contient l'ensemble des vulnérabilités sous forme d'un tableau `vulnerabilities`. Pour permettre une exploitation avancée de ce tableau dans Elasticsearch, notamment la possibilité de requêter chaque vulnérabilité de manière indépendante tout en préservant son rattachement au rapport global, le champ doit être indexé avec le type `nested`. Comme indiqué dans la documentation officielle d'Elasticsearch,<sup>39</sup> ce type simule une relation interne de type Parent-Fils et crée, pour chaque élément du tableau, une structure isolée conservant sa propre cohérence.

Cependant, ce type `nested` présente une limite importante : les outils de visualisation, tels que Kibana, ne peuvent pas directement afficher ni manipuler efficacement les éléments internes d'un `nested`.<sup>40</sup> Les visualisations nécessitent des documents plats, où chaque vulnérabilité existe comme entité indépendante.

Une première solution envisagée consistait à utiliser un processeur d'ingestion capable d'émettre plusieurs documents (Parent et Enfants) lors du passage dans le pipeline. Toutefois, la version d'Elasticsearch utilisée ne fournit pas encore le processeur `emit()`, rendant impossible la génération automatique de documents multiples au sein du pipeline.<sup>es:es\_ingest\_pipelines</sup> Au-delà de cette contrainte technique, la génération externalisée des documents nous permet également de conserver un contrôle strict sur la structure, la cohérence et la traçabilité du flux de données.

C'est dans ce contexte qu'a été introduit le script `split_reports.py`. Celui-ci lit le `report.json` normalisé et produit un fichier unique au format NDJSON contenant :

- un **Document Parent** conservant intégralement le tableau `vulnerabilities[]` pour les usages Watcher et les analyses globales ;
- un ensemble de **Documents Enfants**, chacun représentant une vulnérabilité individuelle optimisée pour les visualisations Kibana.

Chaque document, Parent ou Enfant, porte le même identifiant `report_id`, garantissant la corrélation entre les deux niveaux. Le choix d'un découpage en amont, avant ingestion, offre

un contrôle total sur la structuration, l'ordre des documents et le respect du schéma attendu côté Elasticsearch, ce qui en fait une solution robuste et maîtrisée.

### 5.2.2 Collecte (Filebeat)

Le Bloc B du schéma illustre le rôle de **Filebeat**.<sup>14</sup> Filebeat est un Agent (*Data Shipper*) de la famille Elastic Beats. Son rôle est de collecter les fichiers `report.json` normalisés et de les expédier à Elasticsearch.

#### Configuration de Filebeat (`filebeat.yml`)

La configuration de Filebeat (`filebeat.yml`) est optimisée pour ce flux de travail :

- **Scan** : L'agent utilise un **Input** (une entrée) de type `log`<sup>23</sup> pour surveiller les répertoires de sortie du script Python. Trois inputs séparés sont définis (Snyk, Trivy, SonarQube).
- **exclude\_files** : `[ '.tmp$' ]` : Cette instruction, appartenant à l'Input `log`,<sup>23</sup> est vitale. Elle garantit l'**écriture atomique** (voir 5.2.6) en ordonnant à Filebeat d'ignorer les fichiers temporaires.
- **scan\_frequency** : `10s` : Paramètre de l'Input `log`<sup>23</sup> qui assure une haute réactivité pour une ingestion rapide post-build.
- **Parsing** : Filebeat utilise le **Processeur intégré json**<sup>24</sup> (configuré via `json.keys_under_root: true`) pour parser le format NDJSON. Cela transforme le texte brut en un objet JSON structuré.
- **Envoi** : L'Output (la sortie) `elasticsearch`<sup>25</sup> est configuré pour envoyer les données JSON structurées directement à Elasticsearch, en spécifiant le pipeline d'ingestion approprié (ex : `pipeline: 'snyk_pipeline'`).

#### Tableau des Fonctionnalités Techniques (Filebeat)

| Composant                                 | Provenance (Nature)             | Rôle dans l'architecture                                                      |
|-------------------------------------------|---------------------------------|-------------------------------------------------------------------------------|
| <code>setup.template.enabled:false</code> | Paramètre (Setup Filebeat)      | Désactivé pour laisser l'Index Template Composable gérer le Mapping.          |
| Input (type <code>filestream</code> )     | Composant (Input Filebeat)      | Définit quels fichiers surveiller ( <code>paths</code> ) et comment les lire. |
| <code>exclude_files</code>                | Paramètre (Input Filebeat)      | Supporte la stratégie d'écriture atomique en ignorant les <code>.tmp</code> . |
| <code>json</code> (Processor)             | Composant (Processeur Filebeat) | Parse les lignes de texte en tant qu'objets JSON (NDJSON).                    |
| Output ( <code>elasticsearch</code> )     | Composant (Output Filebeat)     | Définit où (hôte ES) et comment (via quel pipeline) envoyer les données.      |

TABLE 5.1 – Rôles des composants Filebeat

## 5.3 Pilier 2 : Métriques d'Infrastructure (Metricbeat)

Pour garantir la stabilité du pipeline CI/CD (gourmand en ressources) et de l'application, nous avons déployé l'agent **Metricbeat**.

### 5.3.1 Configuration Stratégique des Modules

Contrairement à une approche "boîte noire", nous avons configuré Metricbeat pour qu'il comprenne le contexte de conteneurisation de notre infrastructure Docker (cf. Annexe B.1).

- **Surveillance Système (module: system)** : Nous collectons toutes les 10 secondes les indicateurs vitaux de l'hôte (CPU par cœur, mémoire, I/O disque). Cela nous permet de détecter si la VM hôte sature lors d'un build Jenkins parallèle.
- **Surveillance Docker (module: docker)** : C'est la configuration clé.<sup>26</sup> En connectant Metricbeat au socket Docker, l'agent interroge directement le démon. **L'apport** : Cela nous permet d'obtenir la consommation de ressources *par conteneur*. Nous pouvons ainsi distinguer si un pic de charge provient du conteneur `sonarqube` (analyse lourde) ou de l'application `babyfoot-backend`.

### 5.3.2 Enrichissement Contextuel

Pour que les métriques soient exploitables dans Kibana, nous utilisons des processeurs (`processors`) qui enrichissent la donnée brute avant son indexation (cf. Annexe B.1) :

- **add\_docker\_metadata** : Ce processeur ajoute automatiquement les labels, les noms et les IDs des conteneurs aux événements. **Pourquoi ?** Sans cela, nous aurions juste des ID de processus anonymes. Avec cet enrichissement, nous pouvons filtrer les graphiques par `container.name: babyfoot-backend`.

## 5.4 Pilier 3 : Traces Applicatives et RUM (Elastic APM)

Afin de mesurer l'expérience utilisateur et la performance interne de l'application *Babyfoot Championship*, nous avons mis en place une instrumentation complète de la stack ("Full-Stack Observability").

### 5.4.1 Instrumentation du Backend (Java Agent)

Le backend étant une application Spring Boot, nous avons opté pour une instrumentation via **Java Agent**, qui ne nécessite aucune modification du code source ("Zero-Code Change").<sup>27</sup>

**Mise en œuvre** : L'agent est injecté au démarrage du conteneur via le fichier `docker-compose.yml`. Nous attachons l'agent à la JVM et définissons les paquets à tracer

spécifiquement (`com.babyfoot`) pour focaliser l'analyse sur notre code métier (cf. Annexe B.2).

### 5.4.2 Instrumentation du Frontend (RUM Agent)

Pour capturer l'expérience utilisateur réelle (chargement des pages, erreurs JS), nous avons intégré l'agent **RUM (Real User Monitoring)** dans l'application React.<sup>28</sup>

**Configuration :** L'initialisation se fait dans le code source frontend. Un point crucial est la configuration des origines de traçage distribué (cf. Annexe B.3). **Pourquoi ?** Par défaut, pour des raisons de sécurité (CORS), le navigateur ne propage pas les en-têtes de traçage vers des domaines tiers. Cette configuration autorise explicitement l'agent RUM à marquer les requêtes à destination de notre API Backend.

### 5.4.3 La Continuité de la Trace (Distributed Tracing)

La valeur ajoutée majeure de cette implémentation est la **Trace Distribuée**. Pour qu'une lenteur affichée sur le Frontend puisse être liée à une requête SQL lente sur le Backend, il faut passer un "bâton de relais" (Trace ID) via les en-têtes HTTP standards (W3C Trace Context).<sup>29</sup>

**Mécanisme technique :** Nous avons configuré un intercepteur sur le client HTTP `axios`. À chaque appel API, cet intercepteur injecte automatiquement l'en-tête `traceparent` standardisé (cf. Annexe B.4).

### 5.4.4 Suivi des Parcours Utilisateur (Custom Transactions)

Au-delà des métriques automatiques, nous avons instrumenté la navigation pour suivre les changements d'onglets comme des transactions métiers distinctes (cf. Annexe B.5). Cela permet de quantifier précisément quelle fonctionnalité (Scoreboard, New Match) est la plus utilisée.

## 5.5 Ingestion, Stockage et Indexation (Elasticsearch)

Une fois les documents Parent et Enfant générés, ils sont transmis à Elasticsearch via Fi-lebeat. Leur traitement repose sur deux mécanismes fondamentaux : les Ingest Pipelines et le Mapping.

### 5.5.1 Les Ingest Pipelines (Le Cerveau de l'Ingestion)

Les documents sont d'abord reçus par un Nœud Ingest d'Elasticsearch, qui applique le pipeline d'ingestion spécifié<sup>12</sup> (une Ressource API d'Elasticsearch) spécifié. Ce pipeline effectue des transformations légères telles que :

- `rename` : harmonisation des noms de champs des deux types de documents,

- `set` : enrichissement ou assignation de champs spécifiques tels que `doc_type`.

Ce pipeline ne génère aucun document supplémentaire ; il assure uniquement la conformité du flux entrant au modèle attendu par l'indexation.

## 5.6 Ingestion, Stockage et Indexation (Elasticsearch)

Une fois les documents Parent et Enfant générés, ils sont transmis à Elasticsearch via Filebeat. Leur traitement repose sur deux mécanismes fondamentaux : les Ingest Pipelines et le Mapping.

### 5.6.1 Ingest Pipelines

Les documents sont d'abord reçus par un nœud Ingest, qui applique le pipeline spécifié. Ce pipeline effectue des transformations légères telles que :

- `rename` : harmonisation des noms de champs des deux types de documents,
- `set` : enrichissement ou assignation de champs spécifiques tels que `doc_type`.

Ce pipeline ne génère aucun document supplémentaire ; il assure uniquement la conformité du flux entrant au modèle attendu par l'indexation.<sup>41</sup>

### 5.6.2 Template, Mapping et Indexation

L'échec de la génération automatique Parent–Enfant via pipeline renforce l'importance d'un mapping adapté aux deux structures documentaires résultantes du découpage.

- **Document Parent.** Le champ `vulnerabilities` y est mappé en `nested`.<sup>39</sup> Ce choix est indispensable pour exécuter des requêtes cohérentes au niveau de chaque vulnérabilité individuelle sans risquer de mélanger les attributs entre elles. Les `Watchers`<sup>es:watcher</sup> exploitent ce modèle pour effectuer des corrélations temporelles ou déclencher des alertes conditionnelles.
- **Document Enfant.** Chaque vulnérabilité isolée est mappée sur une structure `plate`, où les champs sont explicitement typés (`keyword`, `float`, `boolean`). Cette modélisation permet des visualisations stables et des filtrages fiables dans Kibana.<sup>40</sup>
- **Corrélation.** Le champ `report_id`, partagé par tous les documents issus du même rapport, agit comme clé de jointure logique entre Parent et Enfants.
- **Dynamic Templates.** Pour absorber les variations propres aux différents scanners (Trivy, Bandit, Safety, etc.), des règles de typage dynamique<sup>42</sup> garantissent une catégorisation par défaut en `keyword`.

Une fois le document validé par le template applicable<sup>30</sup> à l'index `pipeline-reports-*`, il est stocké dans l'index journalier correspondant. Le schéma hybride combinant `nested` pour le Parent et documents plats pour les Enfants permet ainsi d'allier richesse contextuelle et finesse d'analyse.

**Tableau des Fonctionnalités Techniques (Elasticsearch Ingestion)**

| Composant                        | Nature                          | Rôle dans l'architecture d'ingestion                                                                                                                          |
|----------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Index Template                   | Configuration (Cluster ES)      | Définit la structure attendue des nouveaux index ( <i>blueprint</i> ) : mapping, settings et règles d'application.                                            |
| Mapping                          | Configuration (Index ES)        | Schéma de stockage : typage des champs, gestion du <code>nested</code> pour le Parent et des champs plats pour les Enfants. Garantit la cohérence analytique. |
| Ingest Pipeline                  | Ressource (Elasticsearch API)   | Point d'entrée des documents. Exécute les transformations légères (normalisation des champs, enrichissement, assignation du <code>doc_type</code> ).          |
| Processors (rename, set, script) | Composants internes du Pipeline | Opérateurs élémentaires appliquant les règles du pipeline : renommage, ajout de champs, logique conditionnelle via scripts.                                   |
| Painless                         | Langage (Scripting Engine)      | Langage utilisé pour les transformations avancées dans les processeurs script ; permet d'intégrer la logique métier dans le pipeline.                         |

TABLE 5.2 – Rôles des composants de l'architecture d'ingestion Elasticsearch

### 5.6.3 Le rôle du champ `doc_type`

Ce champ, défini par le script Python, joue un rôle fondamental :

1. `vulnerability_report` : utilisé par les Watchers pour lire les rapports complets et le tableau `nested` ;
2. `vulnerability_finding` : utilisé dans Kibana pour afficher chaque vulnérabilité individuellement.

Il permet à Elasticsearch de router chaque document vers la bonne logique d'indexation et aux applications clientes d'adapter leur traitement automatiquement.

## 5.7 Conclusion

Cette architecture élimine définitivement l'hétérogénéité des données de sécurité.

En combinant un script Python dédié pour la normalisation profonde (le cœur de la transformation) avec Filebeat et les Ingest Pipelines d'Elasticsearch pour l'ingestion et l'enrichissement

(le cerveau du flux), nous obtenons une chaîne de traitement découplée, robuste et hautement performante.

Le résultat final : un jeu de données unifié au format **ECS**, centralisé dans un index unique, prêt pour la corrélation avancée, les visualisations consolidées et la mise en œuvre fiable du système d'alerting automatisé (Watcher 1 et 2)

## Chapitre 6

# Observabilité et Visualisation : La Tour de Contrôle

### 6.1 Introduction

Après avoir stabilisé le flux d'ingestion via l'architecture normalisée (scripts Python et Filebeat), l'étape suivante consiste à transformer cette masse de données techniques en informations décisionnelles. L'absence de visibilité identifiée dans l'existant (risques O4, O6) nécessite une « Tour de Contrôle » : une suite de **7 Dashboards Kibana** interconnectés. Ces interfaces opérationnelles répondent aux questions spécifiques des différentes parties prenantes (Développeurs, Ops, SecOps, Management) en s'appuyant sur l'architecture parent/enfant des documents Elasticsearch (`doc_type: vulnerability_report` pour les agrégations, `doc_type: vulnerability_finding` pour les analyses détaillées).

### 6.2 Dashboard 1 : Security Unified

#### 6.2.1 Présentation

Ce dashboard, destiné au CISO, aux Tech Leads et aux responsables sécurité, offre une vue immédiate du risque global en montrant la répartition des vulnérabilités par criticité, les services et outils les plus exposés, ainsi que les dix failles critiques nécessitant une intervention urgente. Il repose sur des documents filtrés (`doc_type: vulnerability_finding`, `status open`) et exploite le schéma ECS pour agréger de façon homogène les données multi-outils par sévérité, service et source de scan.

6.2.2 Visualisations

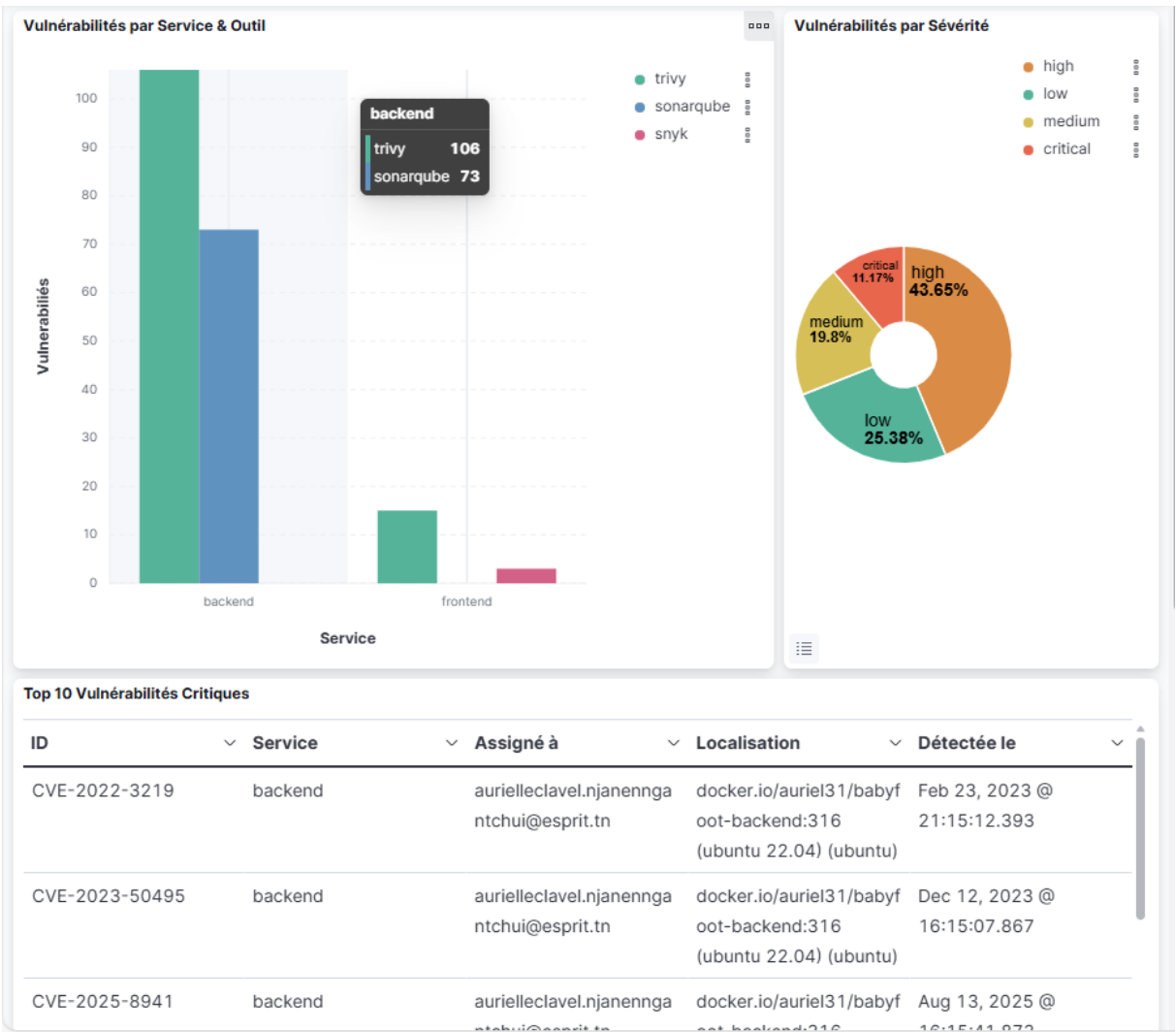


FIGURE 6.1 – Dashboard Security Unified - Vue globale de la posture de sécurité

1. Donut Chart - Vulnérabilités par Sévérité (haut droite)

Répartition visuelle des criticités suivant un code couleur normalisé permettant évaluation immédiate du risque global.

2. Vertical Bar - Vulnérabilités par Service & Outil (haut gauche)

Montre une comparaison efficacité sur le nombre de vulnérabilités détectées par service et par outil de scan, couverture des scanners et priorisation efforts de correction sur services les plus affectés.

3. Data Table - Top 10 Critiques (bas)

Tableau détaillé listant les vulnérabilités les plus critiques avec leurs métadonnées.

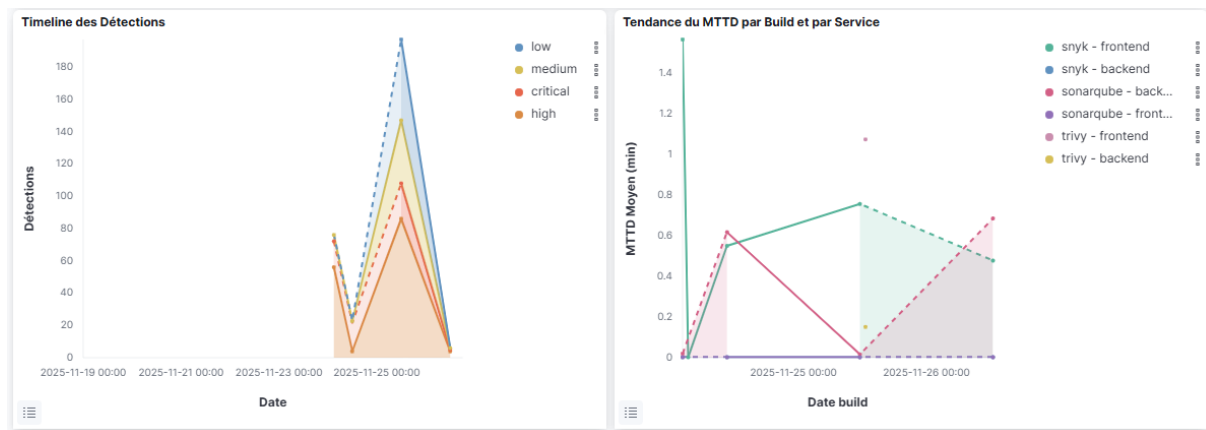


FIGURE 6.2 – Dashboard Security Unified - Vue globale de la posture de sécurité

### 1. Line Chart - Variations du MTDD (A droite)

Cette vue trace la tendance du temps moyen de détection (MTDD) par outil et service au fil du temps. Elle permet de valider la performance du pipeline DevSecOps : si les courbes sont basses et stables, la détection est rapide ; tout pic indique une régression ou un incident ayant ralenti le processus de scan.

### 2. Line Chart - Chronologie des Détections des sévérités (A gauche)

Montre le nombre de détections au fil du temps par niveau de gravité et permet de surveiller la tendance des problèmes de sécurité introduits ou découverts dans le code.

## 6.3 Dashboard 2 : Snyk SCA (Analyse de Composition Logicielle)

### 6.3.1 Présentation

Ce dashboard analyse la sécurité des dépendances Maven et NPM pour aider développeurs et leads à identifier les packages les plus vulnérables, les failles disposant d'exploits publics et celles corrigibles par simple mise à jour. Basé sur les données Snyk filtrées (`tool.name: snyk`) et enrichies avec les champs dédiés (`can_auto_upgrade`, `is_exploitable`), il agrège les vulnérabilités par package afin d'isoler les « racines » du risque. Il transforme ainsi un volume brut de failles en un plan de remédiation ciblé, permettant des corrections massives et une priorisation fondée sur le risque réel plutôt que sur le CVSS théorique.

## 6.3.2 Visualisations

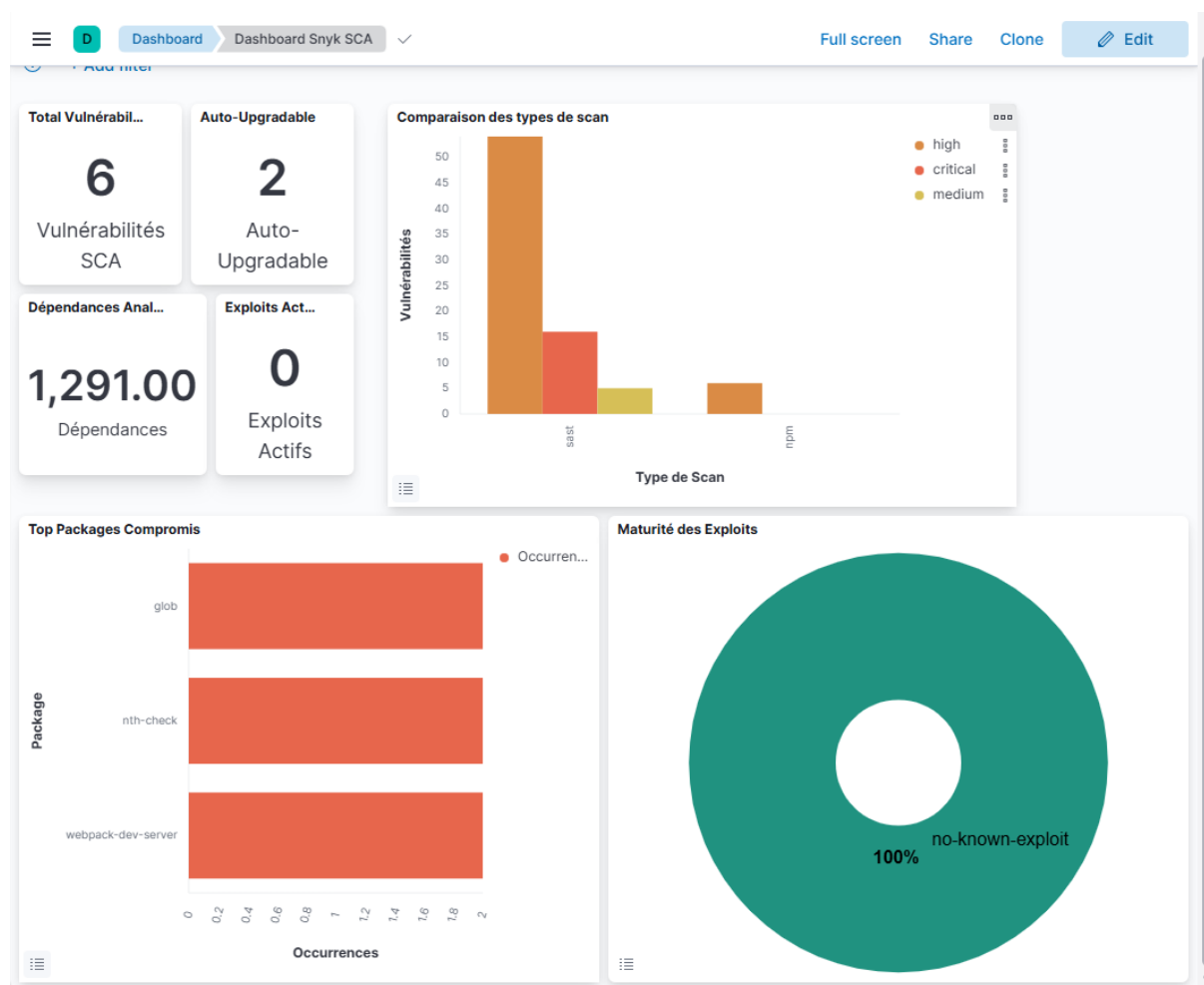


FIGURE 6.3 – Dashboard Snyk SCA - Analyse des dépendances

**1. Quatre Metrics KPI (haut)**

Total vulnérabilités SCA, dépendances analysées, auto-upgradables (indicateur d’effort de remédiation), exploits actifs (criticité opérationnelle). KPI « Auto-Upgradable » quantifie travail simple (npm update/pom.xml) réalisable rapidement.

**3. Horizontal Bar - Top Packages (bas)**

Classement dépendances problématiques par occurrence, révélant racines du problème. Correction d’un package parent élimine vulnérabilités transitives, maximisant impact des interventions.

**4. Pie Chart - Maturité des Exploits (milieu droite)**

Distribution par statut d’exploit (Public/PoC/Aucun). Permet priorisation basée sur criticité opérationnelle : vulnérabilité Medium avec exploit public > vulnérabilité High sans exploit connu.

## 6.4 Dashboard 3 : Trivy Container Security

### 6.4.1 Présentation

Ce dashboard sécurise la couche conteneur en identifiant les images Docker comportant des CVE critiques, les vulnérabilités disposant de correctifs et les packages système les plus exposés. Basé sur les données Trivy filtrées (`tool.name: trivy`) et enrichies avec les métadonnées des images, il segmente les risques par image, layer et type de package. Cette approche permet d'appliquer un Quality Gate strict (« 0 CVE critique »), de générer immédiatement une to-do list exploitable et d'identifier les racines du risque (via targets/packages), facilitant les corrections massives via une simple mise à jour de base image.

### 6.4.2 Visualisations

#### 1. Quatre Metrics KPI (haut gauche)

Total vulnérabilités conteneurs, images scannées, CVE critiques, fix disponibles ( foncé - remédiation immédiate possible).

#### 2. Horizontal Bar - Vulnérabilités par Image (haut droite)

Comparateur de Santé, graphique qui compare nos différentes images logicielles et permet de savoir où concentrer l'effort.

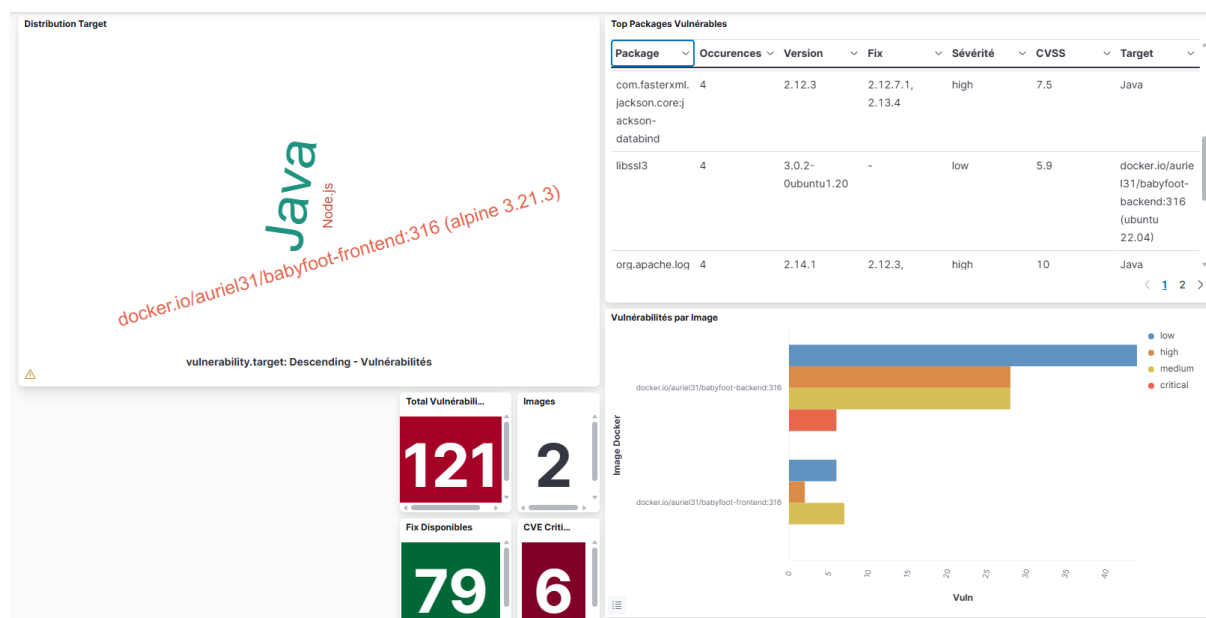


FIGURE 6.4 – Dashboard Trivy Container Security - Sécurité des images Docker

#### 3. Data Table - Top Packages (bas gauche)

Liste détaillée des composants dangereux. exacte, développeur modifie fichier sans recherche. C'est la "To-Do List" technique.

#### 4. La Carte des Sources (bas droite)

Nuage de mots avec taille proportionnelle aux problèmes. Permet d'identifier la racine du mal

## 6.5 Dashboard 4 : SonarQube SAST + Qualité

### 6.5.1 Présentation

Ce dashboard aide les Leads et développeurs à piloter la santé du code en vérifiant l'état du Quality Gate, la répartition des bugs, code smells et vulnérabilités, ainsi que la dette technique par service. Il combine les rapports SonarQube et les findings SAST pour mesurer couverture, duplication et sévérités dans le temps. L'ensemble permet de prévenir l'accumulation de dette, d'orienter les refactorings et d'assurer la robustesse à long terme du code.

### 6.5.2 Visualisations

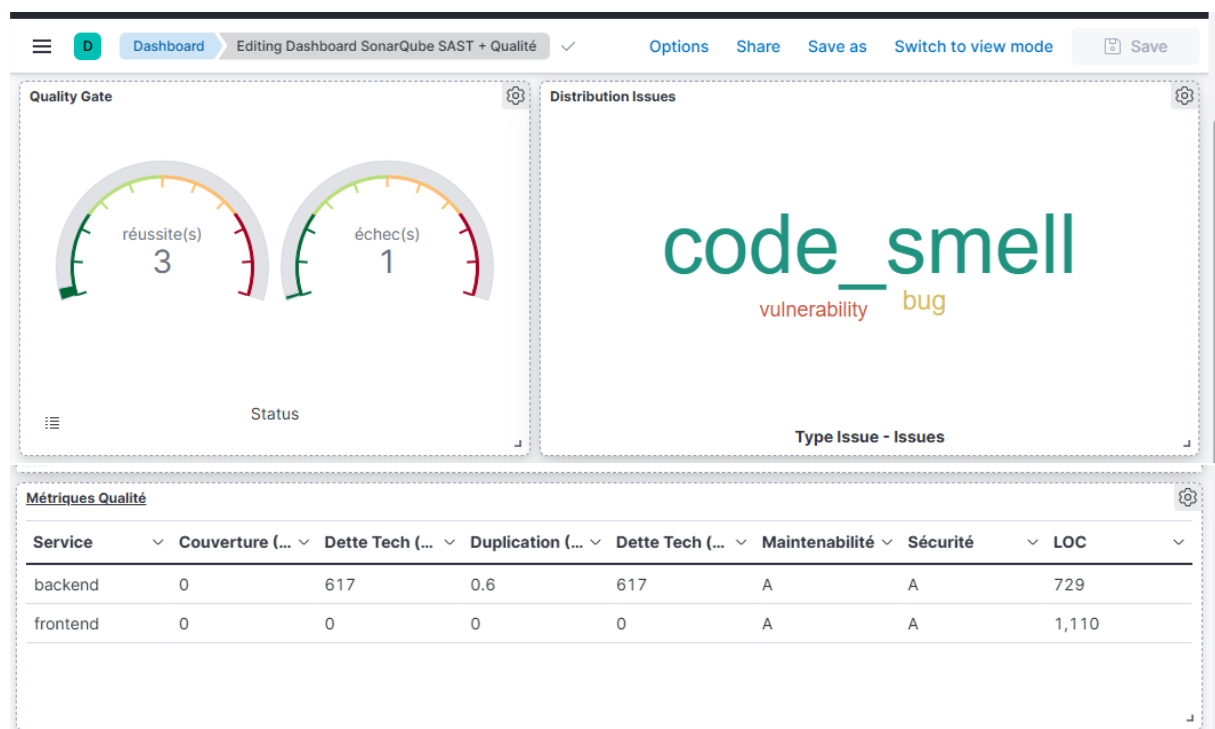


FIGURE 6.5 – Dashboard SonarQube SAST + Qualité - Santé du code

#### 1. Gauge - Quality Gate Status (haut gauche)

Indicateur visuel clé reflétant le résultat de l'analyse : 3 Passed / 1 Failed sur l'historique. Agit comme feu de circulation pour pipeline CI/CD : statut Failed bloque logiquement déploiement, protégeant branche main de toute dégradation majeure.

#### 2. Line Chart La Métrique Économique (Dette Technique par Service)

Mesure l'efficacité des efforts qualité dans le temps. Cette donnée permet de planifier le refactoring. Au lieu de dire "le code est sale", le Lead Dev dit : "Nous avons 617 minutes de dette sur le Backend. Je demande l'allocation de ce temps dans le prochain sprint pour refactorer (Clean Code), ce qui réduira nos coûts futurs." Cela transforme un problème technique en une décision business mesurable.

## 6.6 Dashboard 5 : Infrastructure (Metricbeat)

### 6.6.1 Présentation

Ce dashboard donne aux Ops une vision instantanée de la santé de l'infrastructure en identifiant les services les plus consommateurs de ressources, l'état des conteneurs et les pics de charge liés aux pipelines. Basé sur Metricbeat, il permet de détecter les pannes, anticiper les saturations et ajuster le dimensionnement avant qu'un incident ne survienne.

### 6.6.2 Visualisations

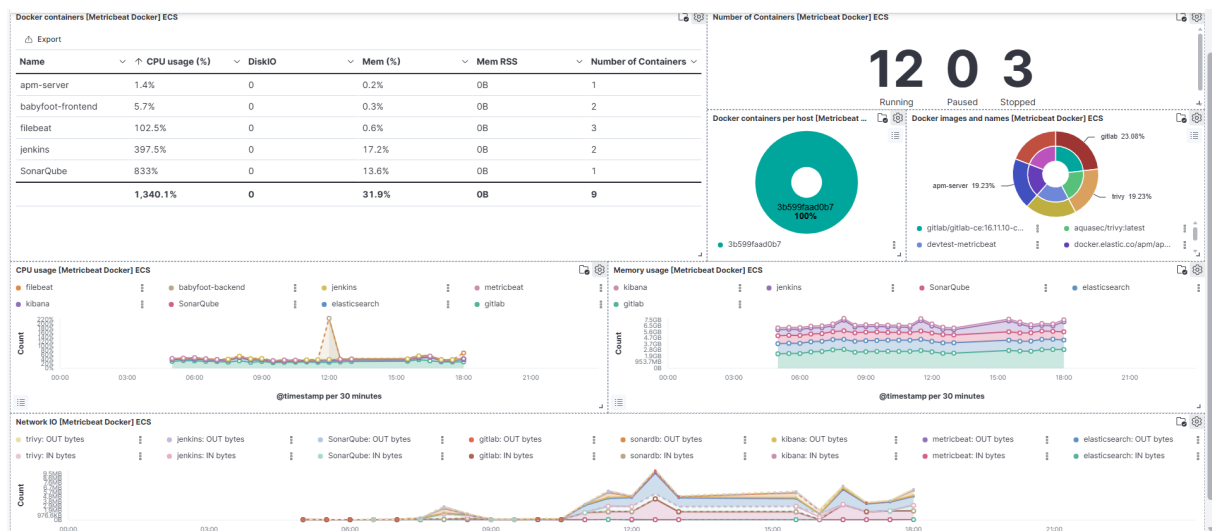


FIGURE 6.6 – Dashboard Metricbeat Infrastructure - Signes vitaux des conteneurs

#### 1. Data Table - Le "Gestionnaire des Tâches" (haut gauche)

Liste détaillée de tous les ouvriers (conteneurs) en poste sur la machine. Identifie immédiatement coupable si système ralentit, sans investigation manuelle.

#### 2. Trois Metrics - État Services (haut droite)

C'est la météo de l'équipe. Cela nous dit instantanément quel(s) service(s) est(sont) : actifs, arrêtés et ont crash. En un coup d'œil : chute brutale Running = panne immédiate.

## 6.7 Dashboard 6 : Performance Applicative (APM/RUM)

### 6.7.1 Présentation

Ce dashboard analyse la performance de l'application de bout en bout en identifiant les goulots d'étranglement entre frontend et backend, les endpoints lents et l'expérience réelle des utilisateurs.

### 6.7.2 Visualisations

#### 1. Core Web Vitals

Le Largest Contentful Paint de 840 ms montre que la page affiche son contenu principal très rapidement. Le Cumulative Layout Shift de 0.000 indique une stabilité visuelle parfaite : aucun élément ne bouge ou ne « saute » pendant le chargement.

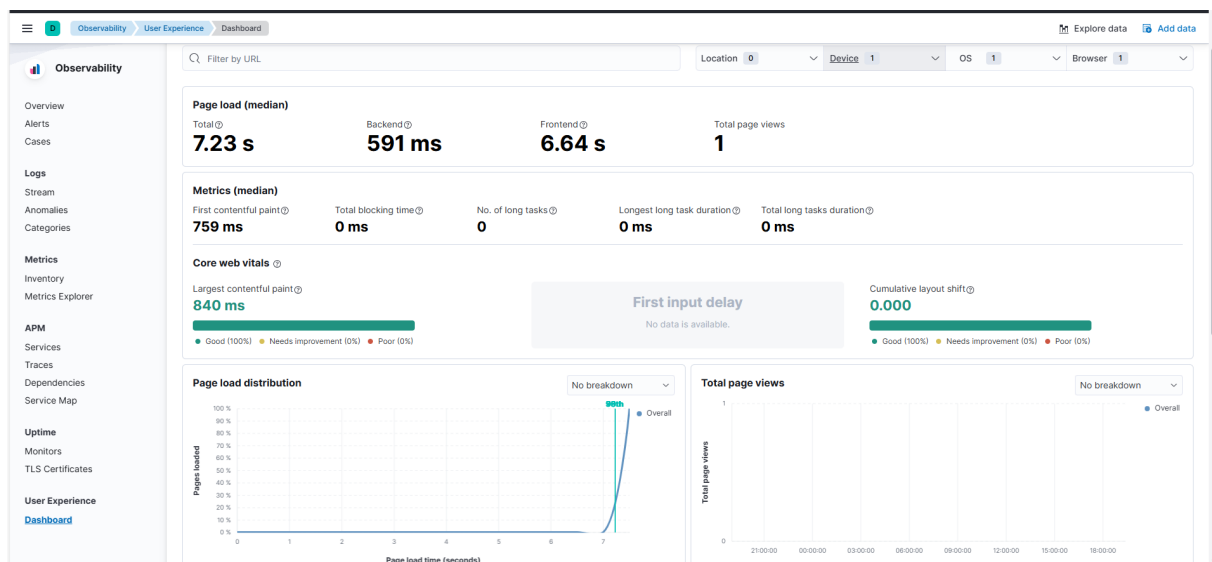


FIGURE 6.7 – Expérience utilisateur détaillée

#### 2. Page Load Breakdown - Frontend vs Backend

La page met 7,23 secondes à se charger, dont seulement 591 ms côté serveur et plus de 6,6 secondes côté frontend. Cela montre clairement que la lenteur vient de l'interface React (images lourdes, scripts), et non du backend ou de la base de données. Sans cette analyse, l'équipe aurait passé des jours à optimiser la mauvaise partie du système.

## 6.8 Conclusion

Ces dashboards offrent une vision claire pour orienter les décisions. Mais cette observabilité reste passive et centrée sur la description. Le chapitre suivant introduit donc l'automatisation nécessaire pour rendre cette surveillance réellement proactive.

## Chapitre 7

# Exploitation Opérationnelle de l'Observabilité : Automatisation du Suivi des Vulnérabilités et Calcul du MTTR

### 7.1 Introduction

Après avoir mis en place la « Tour de Contrôle » offrant une visibilité complète via les dashboards Kibana, l'enjeu est désormais d'exploiter cette observabilité pour automatiser le suivi des vulnérabilités. Ce chapitre décrit l'architecture opérationnelle permettant de transformer les données observées en actions : surveillance continue via Watcher, détection des dérives et calcul du MTTR. L'objectif est d'établir une boucle de rétroaction DevSecOps où la détection, l'alerte et la résolution deviennent automatisées, mesurables et pilotables.

Le système repose sur deux Watchers Elasticsearch fonctionnant en tandem :

- **Le Calculateur (Producteur)** : Un Watcher (`watcher_mark_resolved`) déclenché par Jenkins qui analyse l'état des vulnérabilités et produit un document résumé des métriques de résolution (MTTR).
- **Le Notificateur (Consommateur)** : Un Watcher (`vulnerability_report_watcher`) fonctionnant sur un intervalle régulier, qui lit le résumé du Calculateur et les rapports bruts pour envoyer des alertes triées à Slack.

## CHAPITRE 7. EXPLOITATION OPÉRATIONNELLE DE L'OBSERVABILITÉ : AUTOMATISATION DU SUIVI DES VULNÉRABILITÉS ET CALCUL DU MTTR

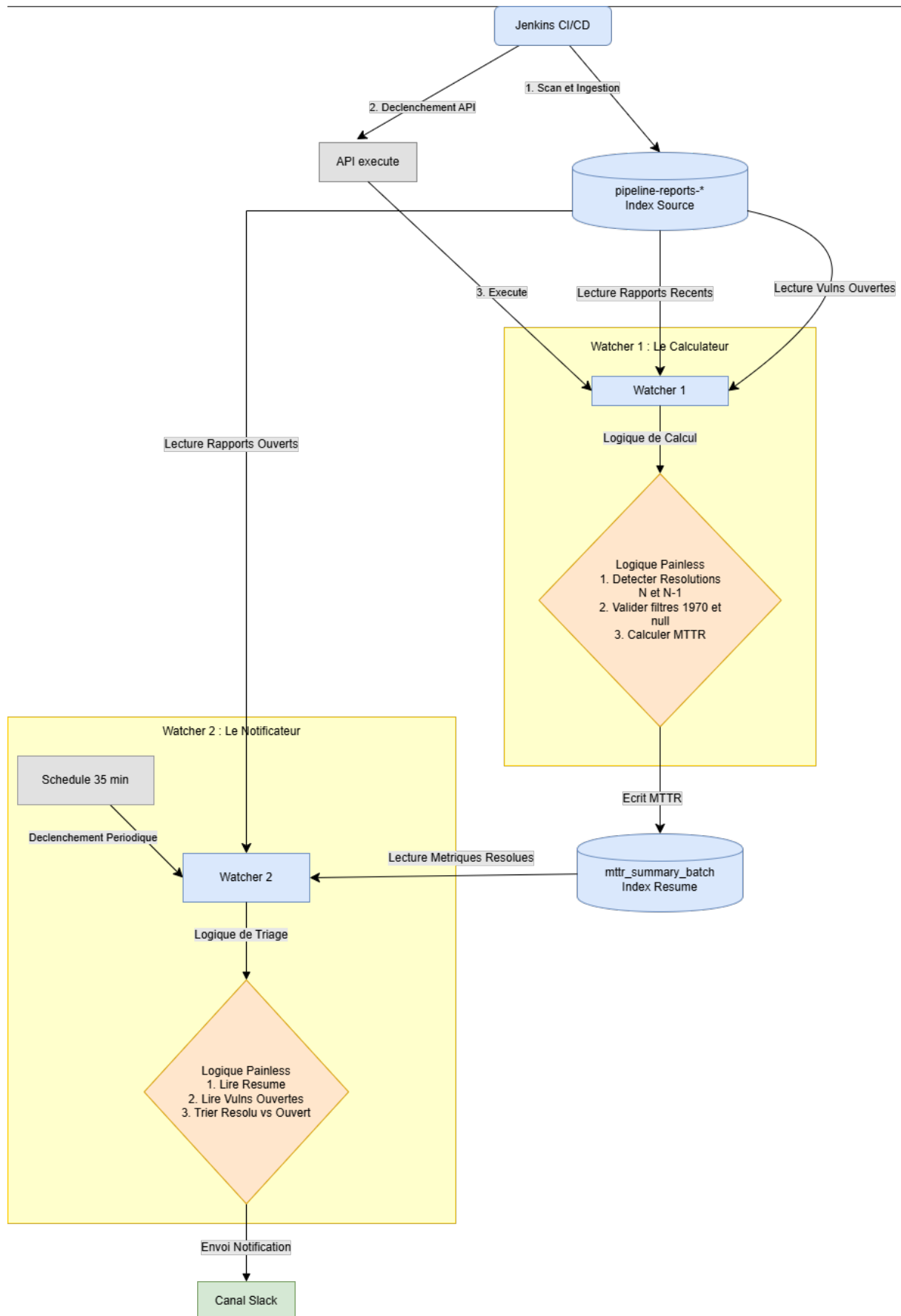


FIGURE 7.1 – Architecture logique du système d'alerting et de calcul MTTR

## 7.2 Watcher 1 : `watcher_mark_resolved` (Le Calculateur de Métriques)

Ce Watcher agit comme le "cerveau" du système. Son unique responsabilité est de détecter les changements d'état (d'ouvert à résolu) et de calculer le MTTR. Il est conçu pour être déclenché manuellement par une API depuis le pipeline Jenkins. Le code JSON complet de ce Watcher est détaillé en [Annexe C.1](#).

### 7.2.1 Logique du Code

Le Watcher utilise un `input` de type `chain` pour exécuter deux requêtes `search` séquentiellement. La première (`latest_report`) récupère le dernier scan par service/outil ; la seconde (`open_vuln_details`) récupère l'historique de toutes les vulnérabilités ouvertes.

Une `condition` d'action vérifie si des résolutions potentielles existent (des vulnérabilités historiques sont absentes du dernier scan). Si la condition est vraie, un `transform` (script `Painless`) est exécuté. Ce script constitue le cœur de la logique :

1. Il compare les deux listes de l'input pour identifier les ID résolus.
2. Il valide que les vulnérabilités résolues étaient bien "ouvertes" (en vérifiant si `resolution_date` est `null` ou `'1970-01-01T00:00:00Z'`).
3. Il calcule le MTTR pour chaque résolution valide.
4. Le script prépare les données pour l'action webhook en les retournant dans le payload.

Enfin, l'action webhook effectue un `_update` avec `upsert` sur le document résumé (`resolved_report`).

## 7.2.2 Fonctionnalités Techniques

| Terme Clé                     | Définition (selon la Documentation Officielle)                                                                                                                                            |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>input</code>            | Définit les données que le Watcher doit charger dans son payload avant l'exécution. C'est la source de données de l'alerte.                                                               |
| <code>chain (Input)</code>    | Un type d'input qui permet d'exécuter plusieurs inputs l'un après l'autre. Le résultat de chaque input est ajouté au payload.                                                             |
| <code>search (Input)</code>   | Un type d'input qui exécute une requête Elasticsearch (Query DSL) et stocke le résultat (la réponse JSON) dans le payload.                                                                |
| <code>aggs/buckets</code>     | Les agrégations sont utilisées dans l'input search pour regrouper les données. Les buckets sont les "contenants" de données résultant de cette agrégation.                                |
| <code>condition</code>        | Un script ou une comparaison qui contrôle si les actions doivent être exécutées. Il doit retourner <code>true</code> pour que le Watcher continue.                                        |
| <code>ctx.payload</code>      | L'objet de contexte ( <code>ctx</code> ) contient les variables d'exécution. <code>ctx.payload</code> est la variable spécifique où sont stockés les résultats du bloc input.             |
| <code>transform</code>        | Un script (généralement Painless) qui permet de modifier, restructurer ou calculer de nouvelles données à partir du <code>ctx.payload</code> avant qu'il ne soit utilisé par les actions. |
| <code>action</code>           | L'opération que le Watcher doit effectuer si la condition est remplie (ex : envoyer un email, un webhook, ou écrire dans un index).                                                       |
| <code>webhook (Action)</code> | Un type d'action qui envoie une requête HTTP (POST, PUT, etc.) à une URL spécifiée. Utilisé ici pour appeler l'API <code>_update</code> d'Elasticsearch.                                  |
| <code>lang: painless</code>   | Spécifie que le langage de script utilisé est "Painless", le langage de script par défaut, sécurisé et performant d'Elasticsearch.                                                        |

TABLE 7.1 – Termes Clés et Définitions (Watcher 1)

## 7.3 Watcher 2 : vulnerability\_report\_watcher (Le Notificateur Slack)

Ce Watcher est le "consommateur" des données produites par le Watcher 1. Son rôle est de fournir des notifications claires et triées aux équipes.

### 7.3.1 Logique du Code

Ce Watcher s'exécute selon un `trigger` (déclencheur) de type `schedule` (toutes les 35 minutes). Comme le Watcher 1, il utilise un `input` de type `chain` :

- **`open_reports`** : Récupère les 10 rapports les plus récents contenant des vulnérabilités "open".
- **`resolved_report`** : Récupère le document parent (`resolved_report`) produit par le Watcher 1.

Une `condition` de type `compare` vérifie si le nombre total de documents ouverts trouvés est supérieur à zéro. Si la condition est vraie, un `transform` (script Painless) est exécuté :

1. Il lit les données de `resolved_summary` (les résolutions) dans un `Map` (`closed_vulns`).
2. Il itère sur les vulnérabilités des `open_reports`.
3. **Logique de Triage** : Il compare chaque vulnérabilité "ouverte" à la liste `closed_vulns`. Si l'ID est dans `closed_vulns`, il est formaté comme "Résolu". Sinon, il est formaté comme "Encore Ouvert".
4. Il construit le message Slack final (`final_msg`) avec deux sections distinctes.

L'action `webhook` envoie le `final_msg` (stocké dans le `payload` par le `transform`) à l'URL Slack.



FIGURE 7.2 – Notifications Slack - Canal Security

### 7.3.2 Fonctionnalités Techniques

| Terme Clé                        | Définition (selon la Documentation Officielle)                                                                                                                      |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>trigger</code>             | L'objet qui définit quand le Watcher doit s'exécuter. Le type <code>schedule</code> utilise une expression interval (ex : 35m).                                     |
| <code>input (chain)</code>       | (Voir Watcher 1).                                                                                                                                                   |
| <code>ctx.payload</code>         | (Voir Watcher 1).                                                                                                                                                   |
| <code>condition (compare)</code> | Un type de condition simple qui effectue une comparaison (ex : <code>gt - supérieur à</code> ) entre une valeur du <code>ctx.payload</code> et une valeur statique. |
| <code>transform</code>           | (Voir Watcher 1) Utilisé ici pour le triage et le formatage du message Slack.                                                                                       |
| <code>action (webhook)</code>    | (Voir Watcher 1) Utilisé ici pour envoyer une charge utile (le message) à un service externe (Slack).                                                               |
| <code>lang: painless</code>      | (Voir Watcher 1).                                                                                                                                                   |

TABLE 7.2 – Termes Clés et Définitions (Watcher 2)

## 7.4 Analyse des Métriques MTTR : Résultats et Interprétation

### 7.4.1 Méthodologie

Une analyse via agrégations `nested` a été conduite sur les données résumées. L'agrégation temporelle (`date_histogram`) a permis d'obtenir un MTTR moyen par jour. Le code complet de la requête est présenté en [Annexe C.2](#).

### 7.4.2 Résultats observés

L'exécution de la requête a retourné 152 documents de vulnérabilités résolues répartis sur trois jours actifs (09, 10, 11 novembre) :

| Date        | Nombre de Résolutions | MTTR Moyen (heures) |
|-------------|-----------------------|---------------------|
| 08 novembre | 0                     | -                   |
| 09 novembre | 74                    | 896,16              |
| 10 novembre | 3                     | 32,13               |
| 11 novembre | 75                    | 54,60               |

TABLE 7.3 – Répartition temporelle des résolutions et MTTR moyen

## CHAPITRE 7. EXPLOITATION OPÉRATIONNELLE DE L'OBSERVABILITÉ : AUTOMATISATION DU SUIVI DES VULNÉRABILITÉS ET CALCUL DU MTTR

### Le Pic Initial du 09 Novembre : nettoyage de la dette historique

Le MTTR de 896h ( 37 jours) correspond au traitement massif de vulnérabilités accumulées avant l'instauration du pipeline DevSecOps. Ce phénomène est :

- normal,
- attendu,
- caractéristique d'une entrée en régime d'un pipeline de sécurité.

### La Stabilisation des 10-11 Novembre (32h-54h)

Les valeurs chutent immédiatement (32h à 54h). Cela illustre :

- un feedback plus court,
- une appropriation progressive du pipeline par les développeurs,
- une dette de sécurité qui cesse de croître.

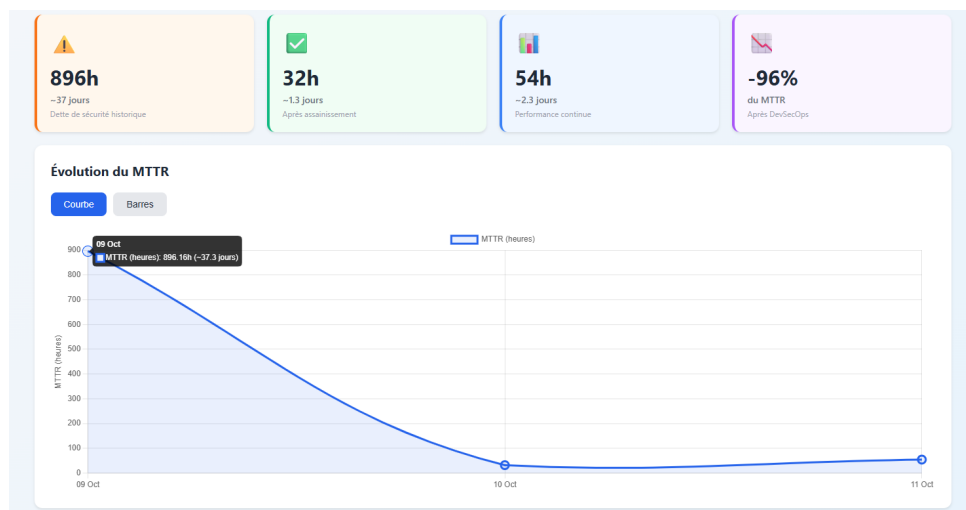


FIGURE 7.3 – Évolution du Temps Moyen de Résolution (MTTR) sur les trois jours actifs

### 7.4.3 Contexte du Proof of Concept (POC)

Les résultats sont issus d'un (PoC) où le comportement de l'équipe de développement a volontairement été simulé de manière pessimiste :

- corrections tardives,
- manque de sensibilisation initiale,
- priorisation limitée de la sécurité.

Ce scénario reflète des environnements réels où DevSecOps n'est pas encore culturellement intégré. Malgré cette simulation "non optimale", le système montre :

- une résilience,

- une capacité à ramener la sécurité à un état stable,
- et une amélioration continue du MTTR.

### 7.5 Conclusion

L'architecture à deux Watchers forme une boucle DevSecOps complète : le premier garantit la précision des métriques et la détection des résolutions, tandis que le second génère des alertes Slack directement exploitables. Combinée à la « Tour de Contrôle » du chapitre précédent, cette automatisation transforme l'observabilité en action. Même dans un scénario défavorable, les résultats montrent que ce mécanisme automatise l'assainissement de la dette, améliore la réactivité et renforce la stabilité de la plateforme. Ce chapitre illustre ainsi le passage décisif de la visualisation à la réaction automatisée dans une démarche DevSecOps.

# Conclusion Générale

Ce projet de fin d'études avait pour objectif de concevoir et de déployer un écosystème DevSecOps complet au sein de l'unité Information Risk Management d'Eneo Cameroon S.A., dans un contexte initial marqué par l'absence de CI/CD, la prédominance des déploiements manuels et une faible maturité en matière de sécurité applicative. L'ensemble de la démarche entreprise, depuis l'analyse de l'existant jusqu'à l'évaluation finale de la plateforme construite, s'est inscrit dans une logique progressive mêlant conception, implémentation, automatisation et mesure continue, conformément aux orientations annoncées dans l'introduction générale.

Les résultats obtenus démontrent que la mise en œuvre d'un pipeline CI/CD sécurisé, associé à une architecture d'observabilité et d'alerting entièrement automatisée, répond efficacement aux problématiques initialement identifiées : manque de visibilité sur les vulnérabilités, inexistence d'un processus de contrôle continu, lenteur des traitements manuels et absence de données fiables pour orienter la décision. La plateforme délivrée permet désormais d'industrialiser SAST, SCA et scans de conteneurs, de centraliser les informations dans un modèle parent-enfant cohérent et d'assurer un suivi structuré des vulnérabilités grâce à un système proactif de Watchers Elasticsearch. Elle constitue ainsi une référence interne démontrant qu'une transition DevSecOps est possible même dans un environnement budgétairement contraint.

La réalisation de ce projet n'a toutefois pas été exempte de difficultés. Parmi les principaux défis rencontrés figurent la nécessité d'intégrer exclusivement des solutions open-source, la complexité du typage et du mapping Elasticsearch (notamment autour du champ nested), l'absence d'automatisation préexistante, ainsi que la gestion d'un environnement hétérogène peu documenté. Ces obstacles ont exigé une forte capacité d'adaptation, une veille technique constante et une rigueur méthodologique dans la conception du pipeline et dans l'ingestion des données.

Sur le plan des apports, ce travail m'a permis de développer des compétences techniques approfondies en CI/CD, Sécurité Applicative, orchestration Docker, pipelines Jenkins, modélisation de données Elasticsearch, scripting Python et automatisation d'alerting avancée. Au-delà de l'aspect technique, ce projet m'a également apporté une meilleure maîtrise de la gestion de projet, de la documentation technique, de l'analyse des besoins et de la communication avec les équipes opérationnelles de l'entreprise.

Enfin, plusieurs perspectives d'amélioration se dégagent naturellement de cette expérience. Il serait pertinent d'étendre la plateforme à d'autres applications internes, d'intégrer des méca-

nismes IaC (Infrastructure as Code), d'automatiser la remédiation de certains incidents à faible impact ou encore d'ajouter des contrôles DAST et de la fuzzing avancée pour mesurer la robustesse des API exposées. À plus long terme, ce travail ouvre également la voie à des recherches sur l'utilisation de modèles d'intelligence artificielle pour la corrélation des vulnérabilités ou la prédiction des comportements de sécurité.

Cette étude constitue ainsi une première étape structurante dans la transformation DevSecOps d'Eneo Cameroon S.A., offrant un cadre reproductible et évolutif qui pourra servir de base à de futurs projets internes ou académiques.

# Bibliographie et Webographie

## Webographie

- [1] OWASP FOUNDATION. *OWASP Secure by Design Framework*. 2024. URL : <https://owasp.org/www-project-secure-by-design-framework/>.
- [2] *Pipeline Syntax and Environment Variables*. Jenkins Documentation. 2024. URL : <https://www.jenkins.io/doc/book/pipeline/>.
- [3] *Documentation officielle Jenkins*. Jenkins. 2024. URL : <https://www.jenkins.io/doc/>.
- [4] *Managing Credentials*. Jenkins Documentation. 2024. URL : <https://www.jenkins.io/doc/book/using/using-credentials/>.
- [5] *Concepts – Architecture*. SonarQube Documentation. 2024. URL : <https://docs.sonarqube.org/latest/architecture/architecture-integration/>.
- [6] *Analyzing Source Code with SonarScanner*. SonarQube Documentation. 2024. URL : [https://docs.sonarqube.org/latest/analyzinges\\_index\\_templatesiew/](https://docs.sonarqube.org/latest/analyzinges_index_templatesiew/).
- [7] *Software Composition Analysis (SCA)*. Snyk Documentation. 2024. URL : <https://docs.snyk.io/scan-applications/snyk-open-source>.
- [8] *Snyk CLI*. Snyk Documentation. 2024. URL : <https://docs.snyk.io/snyk-cli>.
- [9] *Authenticate the Snyk CLI*. Snyk Documentation. 2024. URL : <https://docs.snyk.io/snyk-cli/authenticate-the-cli-with-your-account>.
- [10] *Scanning Vulnerabilities in Containers*. Trivy Documentation. 2024. URL : <https://aquasecurity.github.io/trivy/>.
- [11] *Elastic Stack Overview*. Elastic. 2024. URL : <https://www.elastic.co/guide/en/elastic-stack/current/index.html>.
- [12] *Ingest pipelines*. Elasticsearch Documentation. 2024. URL : <https://www.elastic.co/guide/en/elasticsearch/reference/current/ingest.html>.
- [13] *Creating Dashboards and Visualizations*. Kibana Documentation. 2024. URL : <https://www.elastic.co/guide/en/kibana/current/dashboard.html>.

- [14] *Filebeat Overview*. Elastic Beats Documentation. 2024. URL : <https://www.elastic.co/guide/en/beats/filebeat/current/filebeat-overview.html>.
- [15] *Metricbeat Reference*. Elastic Beats Documentation. 2024. URL : <https://www.elastic.co/guide/en/beats/metricbeat/current/index.html>.
- [16] *Docker Compose Overview*. Docker Documentation. 2024. URL : <https://docs.docker.com/compose/>.
- [17] *Docker Networking Overview*. Docker Documentation. 2024. URL : <https://docs.docker.com/network/>.
- [18] *Self-managed GitLab*. GitLab Documentation. 2024. URL : <https://docs.gitlab.com/ee/>.
- [19] *Webhooks*. GitLab Documentation. 2024. URL : <https://docs.gitlab.com/ee/user/project/integrations/webhooks.html>.
- [20] *Jenkins User Handbook : Overview*. Jenkins Documentation. 2024. URL : <https://www.jenkins.io/doc/book/>.
- [21] *Elastic Common Schema (ECS) Reference*. Elastic Documentation. 2024. URL : <https://www.elastic.co/guide/en/ecs/current/ecs-field-reference.html>.
- [22] *The Python Standard Library (os.rename)*. Python Software Foundation. 2024. URL : <https://docs.python.org/3/library/os.html#os.rename>.
- [23] *Filebeat Input : Log*. Elastic Beats Documentation. 2024. URL : <https://www.elastic.co/guide/en/beats/filebeat/current/filebeat-input-log.html>.
- [24] *Filebeat Processor : Decode JSON fields*. Elastic Beats Documentation. 2024. URL : <https://www.elastic.co/guide/en/beats/filebeat/current/decode-json-fields.html>.
- [25] *Filebeat Output : Elasticsearch*. Elastic Beats Documentation. 2024. URL : <https://www.elastic.co/guide/en/beats/filebeat/current/elasticsearch-output.html>.
- [26] *Metricbeat Reference - Docker Module*. Consulté le 20 Novembre 2025. Elastic Documentation. 2025. URL : <https://www.elastic.co/guide/en/beats/metricbeat/current/metricbeat-module-docker.html>.
- [27] *Java Agent Reference - Automatic Setup*. Consulté le 21 Novembre 2025. Elastic Documentation. 2025. URL : <https://www.elastic.co/guide/en/apm/agent/java/current/setup-javaagent.html>.

- [28] *APM Real User Monitoring (RUM) JavaScript Agent*. Consulté le 21 Novembre 2025. Elastic Documentation. 2025. URL : <https://www.elastic.co/guide/en/apm/agent/rum-js/current/index.html>.
- [29] *Trace Context - W3C Recommendation*. Consulté le 21 Novembre 2025. W3C. 2025. URL : <https://www.w3.org/TR/trace-context/>.
- [30] *Index templates*. Elasticsearch Documentation. 2024. URL : <https://www.elastic.co/guide/en/elasticsearch/reference/current/index-templates.html>.
- [31] SCALED AGILE, INC. *Case Study : Standard Bank – Implementing SAFe and DevOps for Agile in Business*. SAFe Framework. 2019. URL : [https://scaledagile.com/case\\_study/standard-bank/](https://scaledagile.com/case_study/standard-bank/).
- [32] CHEF SOFTWARE. *Standard Bank : Our DevOps Journey*. Chef. 2018. URL : <https://www.chef.io/docs/cheflibraries/case-studies/chef-standard-bank-whitepaper.pdf>.
- [33] MACTORES. *Case Study - Safaricom - Transforming Customer Service with AWS Generative AI and Enhanced Cloud Security*. Mactores Cloud Consulting (Partenaire AWS). 2022. URL : <https://mactores.com/hubfs/AWS%20Generative%20AI%20Competency/Safaricom%20-%20Transforming%20Customer%20Service%20with%20AWS%20Generative%20AI%20and%20Enhanced%20Cloud%20Security%20with%20Mactores.pdf>.
- [34] CYBERRES. *Vodacom Case Study (NetIQ Identity Manager)*. CyberRes (anciennement Micro Focus). 2021. URL : [https://cdn.featuredcustomers.com/CustomerCaseStudydocument/cyberres\\_vodacom\\_654261.pdf](https://cdn.featuredcustomers.com/CustomerCaseStudydocument/cyberres_vodacom_654261.pdf).
- [35] OPENTEXT. *Vodacom : Virtualization Case Study*. OpenText. 2020. URL : <https://www.opentext.com/customers/vodacom>.
- [36] MICRO FOCUS. *MTN South Africa Case Study (ALM and Testing)*. Micro Focus (OpenText). 2021. URL : <https://www.microfocus.com/media/case-study/mtn-south-africa-cs.pdf>.
- [37] OPENTEXT. *MTN South Africa Case Study (Digital Transformation Context)*. OpenText. 2022. URL : <https://www.opentext.com/customers/mtn-south-africa>.
- [38] ELASTIC. *What is Observability?* Consulté le 10 juillet 2025. 2025. URL : <https://www.elastic.co/what-is/observability>.
- [39] ELASTIC. *Nested field type*. Elastic. 2025. URL : <https://www.elastic.co/guide/en/elasticsearch/reference/current/nested.html> (visité le 27/11/2025).

- [40] ELASTIC. *Nested object fields not visible in Kibana visualizations*. Cette référence pointe vers une discussion officielle confirmant la limitation de Kibana sur les champs imbriqués ('nested'). 2024. URL : <https://discuss.elastic.co/t/nested-object-fields-not-visible-in-kibana-visualizations/355058> (visité le 27/11/2025).
- [41] ELASTIC. *Ingest pipelines*. Documentation générale sur les pipelines d'ingestion pour justifier la nécessité d'une solution externe en l'absence de processeur de document multiple intégré. Elastic. 2025. URL : <https://www.elastic.co/guide/en/elasticsearch/reference/current/ingest.html> (visité le 27/11/2025).
- [42] ELASTIC. *Dynamic templates*. Elastic. 2025. URL : <https://www.elastic.co/guide/en/elasticsearch/reference/current/dynamic-templates.html> (visité le 27/11/2025).

# Annexe A

## Détails de Configuration et Scripts (Chapitre 4)

### A.1 Configurations du fichier docker-compose.yml

#### A.1.1 Configuration du service GitLab

```
1 gitlab:
2   image: gitlab/gitlab-ce:16.11.10-ce.0
3   container_name: gitlab
4   hostname: gitlab.kewel.local
5   ports:
6     - "8080:80" # Interface Web
7     - "4443:443" # HTTPS
8     - "2222:22" # SSH pour git clone
9   volumes:
10     - ./gitlab-config:/etc/gitlab
11     - ./gitlab-logs:/var/log/gitlab
12     - ./gitlab-data:/var/opt/gitlab
13   extra_hosts:
14     - "jenkins.kewel.local:172.31.14.7"
15   networks:
16     - devnet
17   restart: unless-stopped
```

Listing A.1 – Service GitLab (docker-compose.yml)

#### A.1.2 Configuration du service Jenkins

```
1 jenkins:
2   build:
3     context: .
```

```
4   dockerfile: Dockerfile
5   container_name: jenkins
6   ports:
7     - "8081:8080" # Interface Web
8     - "50000:50000" # Port pour les agents
9   volumes:
10    - jenkins_home:/var/jenkins_home
11    - ./certs/gitlab.crt:/usr/local/share/ca-certificates/gitlab.crt
12    - ./logs/shared:/shared # Volume partage crucial
13    - /var/run/docker.sock:/var/run/docker.sock
14   extra_hosts:
15     - "gitlab.kewel.local:172.31.14.7"
16   networks:
17     - devnet
18   restart: unless-stopped
19   environment:
20     - JAVA_OPTS=-Xmx2048m -Xms512m -XX:MaxMetaspaceSize=256m
```

Listing A.2 – Service Jenkins (docker-compose.yml)

### A.1.3 Configuration SonarQube et PostgreSQL

```
1   postgres:
2     image: postgres:15-alpine
3     container_name: sonardb
4     environment:
5       - POSTGRES_USER=${POSTGRES_USER}
6       - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
7       - POSTGRES_DB=${POSTGRES_DB}
8     volumes:
9       - sonardb_data:/var/lib/postgresql/data
10    networks:
11      - devnet
12    restart: unless-stopped
13
14   sonarqube:
15     image: sonarqube:9.9.8-community
16     container_name: SonarQube
17     depends_on:
18       - postgres
19     environment:
20       - SONAR_JDBC_URL=jdbc:postgresql://sonardb:5432/sonarqube
21       - SONAR_JDBC_USERNAME=${POSTGRES_USER}
22       - SONAR_JDBC_PASSWORD=${POSTGRES_PASSWORD}
23     ports:
24       - "9000:9000"
```

```
25 volumes:
26   - sonarqube_conf:/opt/sonarqube/conf
27   - sonarqube_data:/opt/sonarqube/data
28   - ./logs/shared:/shared
29 networks:
30   - devnet
31 restart: unless-stopped
```

Listing A.3 – Services SonarQube et DB

### A.1.4 Configuration Elastic Stack (Elasticsearch et Kibana)

```
1  elasticsearch:
2    image: docker.elastic.co/elasticsearch/elasticsearch:7.17.7
3    container_name: elasticsearch
4    environment:
5      - discovery.type=single-node
6      - xpack.security.enabled=true
7      - ELASTIC_PASSWORD=${ELASTIC_PASSWORD}
8      - "ES_JAVA_OPTS=-Xms1g -Xmx1g"
9    volumes:
10     - es_data:/usr/share/elasticsearch/data
11    ports:
12     - "9200:9200"
13    networks:
14     - devnet
15    restart: unless-stopped
16
17  kibana:
18    image: docker.elastic.co/kibana/kibana:7.17.7
19    container_name: kibana
20    volumes:
21     - ./config/kibana/kibana.yml:/usr/share/kibana/config/kibana.yml:ro
22    ports:
23     - "5601:5601"
24    networks:
25     - devnet
26    depends_on:
27     - elasticsearch
28    restart: unless-stopped
```

Listing A.4 – Services Elastic Stack

### A.1.5 Configuration Filebeat

```

1 filebeat:
2   build:
3     context: ./config/filebeat
4     dockerfile: Dockerfile
5   container_name: filebeat
6   user: root
7   command: ["filebeat", "-e", "-d", "publish"]
8   volumes:
9     - ./logs/shared:/pipeline-reports:ro # Lecture seule
10  depends_on:
11    - elasticsearch
12    - kibana
13  networks:
14    - devnet
15  restart: unless-stopped

```

Listing A.5 – Service Filebeat

## A.2 Construction de l'Agent Jenkins Personnalisé

### A.2.1 Dockerfile de l'Agent Jenkins

```

1 FROM jenkins/jenkins:lts
2 USER root
3
4 # Installation de Docker CLI
5 RUN apt-get update && \
6     apt-get install -y docker.io && \
7     usermod -aG docker jenkins
8
9 # Installation de Node.js 18 (pour Snyk + Frontend)
10 RUN curl -fsSL https://deb.nodesource.com/setup_18.x | bash - && \
11     apt-get install -y nodejs
12
13 # Installation de Maven 3.8.4 (pour Backend Java)
14 RUN wget
15     https://archive.apache.org/dist/maven/maven-3/3.8.4/binaries/apache-maven-3.8.4-bin.t
16     && \
17     tar -xzf apache-maven-3.8.4-bin.tar.gz -C /opt && \
18     ln -s /opt/apache-maven-3.8.4/bin/mvn /usr/local/bin/mvn
19
20 # Installation de PHP et Composer (pour projets Laravel)
21 RUN apt-get install -y php php-cli php-mbstring php-xml unzip && \
22     curl -sS https://getcomposer.org/installer | php --
23     --install-dir=/usr/local/bin --filename=composer

```

```
21
22 # Installation de Snyk CLI
23 RUN npm install -g snyk snyk-to-html
24
25 # Installation de Trivy
26 RUN wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key |
    apt-key add - && \
27     echo "deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release
        -sc) main" | tee /etc/apt/sources.list.d/trivy.list && \
28     apt-get update && \
29     apt-get install -y trivy
30
31 # Copie du script de normalisation Python
32 COPY normalize_reports_v3.py /usr/local/bin/normalize-reports.py
33 RUN chmod +x /usr/local/bin/normalize-reports.py && \
34     apt-get install -y python3 python3-pip
35
36 USER jenkins
```

Listing A.6 – Dockerfile de l’agent Jenkins

## A.3 Commandes et Scripts de Configuration

### A.3.1 Commandes de déploiement du réseau et des services

```
1 # Creation du reseau
2 docker network create devnet
3
4 # Demarrage de tous les services
5 docker-compose up -d
6
7 # Verification de l'etat
8 docker-compose ps
9
10 # Consultation des logs en temps reel
11 docker-compose logs -f jenkins
```

Listing A.7 – Séquence de déploiement

### A.3.2 Génération des clés SSH pour Jenkins

```
1 # Acces au conteneur Jenkins
2 docker exec -it jenkins bash
3
4 # Generation de la cle (sans passphrase pour l'automatisation)
```

```
5 ssh-keygen -t rsa -b 4096 -C "jenkins@kewel.local" -f  
   /var/jenkins_home/.ssh/id_rsa -N ""  
6  
7 # Affichage de la cle publique (a copier dans GitLab)  
8 cat /var/jenkins_home/.ssh/id_rsa.pub
```

Listing A.8 – Génération SSH

### A.3.3 Test de connexion SSH Jenkins vers GitLab

```
1 # Depuis le conteneur Jenkins  
2 ssh -T git@gitlab.kewel.local -p 2222  
3 # Sortie attendue: Welcome to GitLab, @jenkins!
```

Listing A.9 – Validation SSH

### A.3.4 Configuration DNS locale (/etc/hosts)

```
1 # Ajouter les lignes suivantes:  
2 127.0.0.1 gitlab.kewel.local  
3 127.0.0.1 jenkins.kewel.local
```

Listing A.10 – Fichier hosts

## A.4 Scripts du Pipeline CI/CD

### A.4.1 Structure globale du Jenkinsfile

```
1 pipeline {  
2   agent any  
3   tools {  
4     maven 'Maven 3.8.4'  
5     nodejs 'NodeJS 18'  
6   }  
7   environment {  
8     SNYK_TOKEN = credentials('snyk-api')  
9     REPORTS_DIR = "/shared"  
10    BUILD_ID = "${env.BUILD_NUMBER}"  
11  }  
12  stages {  
13    stage('Checkout & Git Metadata') { /* ... */ }  
14    stage('Build & SAST Backend') { /* ... */ }  
15    stage('Build & SAST Frontend') { /* ... */ }  
16    stage('Containerization') { /* ... */ }
```

```

17     stage('Security Scan Images (Trivy)') { /* ... */ }
18     stage('Push Docker Images') { /* ... */ }
19     stage('Attente Ingestion Filebeat') { /* ... */ }
20 }
21 }

```

Listing A.11 – Squelette du Jenkinsfile

## A.4.2 Stage 1 : Récupération et Métadonnées (Capture T0)

```

1 stage('Checkout & Git Metadata') {
2     steps {
3         script {
4             echo "ETAPE 1: Recuperation du code et metadonnees Git"
5             checkout scm
6
7             // Capture du timestamp T0
8             def commitTime = System.currentTimeMillis()
9             env.CODE_INTRODUCTION_TIME = commitTime
10
11            // Extraction des metadonnees Git
12            env.GIT_COMMIT_HASH = sh(script: "git rev-parse --short HEAD",
13                                     returnStdout: true).trim() ?: 'unknown'
14            env.GIT_BRANCH = sh(script: "git rev-parse --abbrev-ref HEAD",
15                               returnStdout: true).trim() ?: 'main'
16            env.GIT_AUTHOR = sh(script: "git log -1 --pretty=format:'%ae'",
17                               returnStdout: true).trim() ?: 'DevTeam'
18        }
19    }
20 }

```

Listing A.12 – Extraction métadonnées Git

## A.4.3 Stage SAST : Analyse SonarQube Backend

```

1 stage('Build & SAST Backend') {
2     steps {
3         script {
4             dir('BabyFOOT/backend') {
5                 sh 'mvn clean install -DskipTests'
6                 def sonarReportDir =
7                     "${REPORTS_DIR}/build-${BUILD_NUMBER}/sonarqube/scans"
8                 sh "mkdir -p ${sonarReportDir}"
9                 def sonarScanStart = System.currentTimeMillis()
10
11                 withSonarQubeEnv('SonarQube') {

```

```

11         sh """
12             mvn sonar:sonar \
13                 -Dsonar.projectKey=babyfoot-backend \
14                 -Dsonar.projectName="Babyfoot Backend" \
15                 -Dsonar.java.binaries=target/classes \
16                 -Dsonar.sources=src/main/java
17             """
18         sleep(45) // Attente traitement asynchrone
19         def sonarScanEnd = System.currentTimeMillis()
20
21         // Recuperation issues et metriques via API (curl)
22         // ... (voir details implementation)
23
24         // Creation metadonnees JSON
25         // ...
26     }
27 }
28 }
29 }
30 }

```

Listing A.13 – Analyse SAST Backend

### A.4.4 Authentification Snyk et Docker

```

1 environment {
2     SNYK_TOKEN = credentials('snyk-api')
3 }
4 // ...
5 withCredentials([usernamePassword(
6     credentialsId: 'docker-registry-credentials',
7     usernameVariable: 'DOCKER_USER',
8     passwordVariable: 'DOCKER_PASSWORD'
9 )]) {
10     sh 'echo $DOCKER_PASSWORD | docker login -u $DOCKER_USER
11         --password-stdin'
12 }

```

Listing A.14 – Injection des secrets

## Annexe B

# Détails d'Implémentation de l'Observabilité (Chapitre 5)

### B.1 Configuration Metricbeat (Docker & Processors)

*Extrait du fichier metricbeat.yml*

```
1 # Configuration du module Docker
2 - module: docker
3   metricsets:
4     - container
5     - cpu
6     - memory
7     - network
8   hosts: ["unix:///var/run/docker.sock"] # Connexion directe au demon
9   period: 10s
10  enabled: true
11
12 # Processeurs pour l'enrichissement des metadonnees
13 processors:
14   - add_docker_metadata:
15     host: "unix:///var/run/docker.sock"
16     match_source: true
```

Listing B.1 – Configuration du module Docker et des processeurs

### B.2 Injection de l'Agent Java (Backend)

*Extrait du fichier docker-compose.yml*

```
1 # Commande de démarrage du conteneur Backend
2 command: >
3   java -javaagent:/elastic-apm-agent.jar
```

## ANNEXE B. DÉTAILS D'IMPLÉMENTATION DE L'OBSERVABILITÉ (CHAPITRE 5)

```
4 -Delastic.apm.service_name=babyfoot-backend
5 -Delastic.apm.server_urls=http://apm-server:8200
6 -Delastic.apm.environment=development
7 -Delastic.apm.application_packages=com.babyfoot
8 -jar app.jar
```

Listing B.2 – Injection de l'agent Java au démarrage

### B.3 Initialisation RUM (Frontend)

*Extrait du fichier src/apm.js*

```
1 const apm = initApm({
2   serviceName: 'babyfoot-frontend',
3   serverUrl: 'http://localhost:8200',
4   // Autorise la corrélation avec le backend sur le port 8085
5   distributedTracingOrigins: ['http://localhost:8085'],
6   capturePageLoad: true
7 })
```

Listing B.3 – Initialisation de l'agent RUM

### B.4 Intercepteur HTTP pour le Tracing Distribué

*Extrait du fichier src/services/api.js*

```
1 api.interceptors.request.use((config) => {
2   // Demarre une "span" APM pour chaque appel API
3   // et injecte le header traceparent automatiquement
4   const span = apm.startSpan(`${config.method?.toUpperCase()}
5     ${config.url}`, 'http')
6   config.metadata = { span }
7   return config
8 })
```

Listing B.4 – Intercepteur Axios pour injection traceparent

### B.5 Instrumentation de la Navigation (Custom Transaction)

*Extrait du fichier src/App.js*

```
1 const handleTabChange = (tab) => {
2   // Cree une transaction personnalisée pour suivre la navigation
3   apm.startTransaction(`Maps to ${tab}`, 'navigation')
4   apm.addLabels({ current_tab: tab })
5 }
```

## ANNEXE B. DÉTAILS D'IMPLÉMENTATION DE L'OBSERVABILITÉ (CHAPITRE 5)

```
5   setActiveTab(tab)
6 }
```

Listing B.5 – Transaction personnalisée sur changement d'onglet

# Annexe C

## Détails de l'automatisation du suivi de l'observabilité (Chapitre 7)

### C.1 Code Watcher : Détection et Résolution des Vulnérabilités

```
1 {
2   "trigger": {
3     "schedule": {
4       "interval": "30d"
5     }
6   },
7   "input": {
8     "chain": {
9       "inputs": [
10        {
11          "latest_report": {
12            "search": {
13              "request": {
14                "search_type": "query_then_fetch",
15                "indices": [
16                  "pipeline-reports-*"
17                ],
18                "rest_total_hits_as_int": true,
19                "body": {
20                  "size": 0,
21                  "query": {
22                    "bool": {
23                      "filter": [
24                        {
25                          "term": {
26                            "doc_type": "vulnerability_report"
```

## ANNEXE C. DÉTAILS DE L'AUTOMATISATION DU SUIVI DE L'OBSERVABILITÉ (CHAPITRE 7)

```
27         }
28     }
29 ],
30     "must_not": [
31     {
32         "term": {
33             "doc_type": "resolved_report"
34         }
35     }
36 ]
37 }
38 },
39 "aggs": {
40     "by_service_and_tool": {
41         "composite": {
42             "size": 200,
43             "sources": [
44             {
45                 "service": {
46                     "terms": {
47                         "field": "service.name",
48                         "missing_bucket": true
49                     }
50             }
51             },
52             {
53                 "tool": {
54                     "terms": {
55                         "field": "tool.name",
56                         "missing_bucket": true
57                     }
58             }
59             }
60         ]
61     },
62     "aggs": {
63         "latest_scan": {
64             "top_hits": {
65                 "size": 1,
66                 "sort": [
67                 {
68                     "@timestamp": "desc"
69                 }
70             ],
71             "_source": [
72                 "@timestamp",
73                 "vulnerabilities",
```

## ANNEXE C. DÉTAILS DE L'AUTOMATISATION DU SUIVI DE L'OBSERVABILITÉ (CHAPITRE 7)

```
74         "metadata",
75         "service",
76         "tool",
77         "report_id",
78         "build",
79         "git",
80         "pipeline"
81     ]
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 },
92 {
93     "open_vuln_details": {
94         "search": {
95             "request": {
96                 "search_type": "query_then_fetch",
97                 "indices": [
98                     "pipeline-reports-*"
99                 ],
100                 "rest_total_hits_as_int": true,
101                 "body": {
102                     "size": 0,
103                     "query": {
104                         "bool": {
105                             "filter": [
106                                 {
107                                     "term": {
108                                         "doc_type": "vulnerability_report"
109                                     }
110                                 },
111                                 {
112                                     "nested": {
113                                         "path": "vulnerabilities",
114                                         "query": {
115                                             "term": {
116                                                 "vulnerabilities.unified.status": "open"
117                                             }
118                                         }
119                                     }
120                                 }
121                             ]
122                         }
123                     }
124                 }
125             }
126         }
127     }
128 }
```

## ANNEXE C. DÉTAILS DE L'AUTOMATISATION DU SUIVI DE L'OBSERVABILITÉ (CHAPITRE 7)

```
121         ],
122         "must_not": [
123             {
124                 "term": {
125                     "doc_type": "resolved_report"
126                 }
127             }
128         ]
129     },
130 },
131 "aggs": {
132     "nested_vulns": {
133         "nested": {
134             "path": "vulnerabilities"
135         },
136         "aggs": {
137             "open_only": {
138                 "filter": {
139                     "term": {
140                         "vulnerabilities.unified.status": "open"
141                     }
142                 },
143                 "aggs": {
144                     "unique_open_ids": {
145                         "terms": {
146                             "field":
147                                 "vulnerabilities.unified.vulnerability_id",
148                             "size": 5000
149                         },
150                         "aggs": {
151                             "back_to_root": {
152                                 "reverse_nested": {},
153                                 "aggs": {
154                                     "oldest_open_hit": {
155                                         "top_hits": {
156                                             "size": 1,
157                                             "sort": [
158                                                 {
159                                                     "@timestamp": "asc"
160                                                 }
161                                             ],
162                                             "_source": [
163                                                 "service",
164                                                 "tool",
165                                                 "vulnerabilities",
166                                                 "metadata",
167                                                 "report_id",
```

## ANNEXE C. DÉTAILS DE L'AUTOMATISATION DU SUIVI DE L'OBSERVABILITÉ (CHAPITRE 7)

```
167         "build",
168         "git",
169         "pipeline",
170         "@timestamp"
171     ]
172 }
173 }
174 }
175 }
176 }
177 }
178 }
179 }
180 }
181 }
182 }
183 }
184 }
185 }
186 }
187 }
188 ]
189 }
190 },
191 "condition": {
192     "script": {
193         "source": "return
194             ctx.payload.latest_report.aggregations.by_service_and_tool.buckets.size()
195             > 0;",
196         "lang": "painless"
197     }
198 },
199 "actions": {
200     "store_resolved_parent": {
201         "condition": {
202             "script": {
203                 "source": "return ctx.payload.resolved_parent != null &&
204                     ctx.payload.resolved_parent.size() > 0;",
205                 "lang": "painless"
206             }
207         },
208         "webhook": {
209             "scheme": "http",
210             "host": "elasticsearch",
211             "port": 9200,
212             "method": "post",
```

## ANNEXE C. DÉTAILS DE L'AUTOMATISATION DU SUIVI DE L'OBSERVABILITÉ (CHAPITRE 7)

```
210     "path":  
211         "/pipeline-reports-{{ctx.payload.index_date}}/_update/resolved_report",  
212     "params": {},  
213     "headers": {  
214         "Content-Type": "application/json"  
215     },  
216     "auth": {  
217         "basic": {  
218             "username": "elastic",  
219             "password": "elastic"  
220         }  
221     },  
222     "body": ""{"doc": {"@timestamp":  
223         "{{ctx.payload.resolutionDate}}", "doc_type":  
224         "resolved_report", "resolved":  
225         {{#toJson}}ctx.payload.resolved_parent{{/toJson}}}, "doc_as_upsert":  
226         true}""  
227 }  
228 },  
229 "store_resolved_findings": {  
230     "condition": {  
231         "script": {  
232             "source": "return ctx.payload.resolved_findings != null &&  
233                 ctx.payload.resolved_findings.size() > 0;",  
234             "lang": "painless"  
235         }  
236     },  
237     "foreach": "ctx.payload.resolved_findings",  
238     "max_iterations": 5000,  
239     "webhook": {  
240         "scheme": "http",  
241         "host": "elasticsearch",  
242         "port": 9200,  
243         "method": "post",  
244         "path": "/pipeline-reports-{{ctx.payload.index_date}}/_doc",  
245         "params": {},  
246         "headers": {  
247             "Content-Type": "application/json"  
248         },  
249         "auth": {  
250             "basic": {  
251                 "username": "elastic",  
252                 "password": "elastic"  
253             }  
254         },  
255         "body": "{{#toJson}}ctx.payload._value{{/toJson}}"
```

## ANNEXE C. DÉTAILS DE L'AUTOMATISATION DU SUIVI DE L'OBSERVABILITÉ (CHAPITRE 7)

```
251     }
252   },
253   "transform": {
254     "script": {
255       "source": ""
256     def resolved_parent = [];
257     def resolved_findings = [];
258     def latest_reports =
259       ctx.payload.latest_report.aggregations.by_service_and_tool.buckets;
260
261     // ids present in most-recent scan (N)
262     def idsPresentInN = [:];
263     def resolutionDate = null;
264
265     for (def bucket : latest_reports) {
266       def hits = bucket.latest_scan.hits.hits;
267       if (hits != null && hits.size() > 0) {
268         def reportN = hits[0];
269         def tsStr = reportN._source['@timestamp'];
270         if (resolutionDate == null ||
271             java.time.ZonedDateTime.parse(tsStr).isAfter(java.time.ZonedDateTime.parse(resolutionDate))) {
272           resolutionDate = tsStr;
273         }
274         if (reportN._source.vulnerabilities != null) {
275           for (def vuln : reportN._source.vulnerabilities) {
276             if (vuln.unified != null && vuln.unified.vulnerability_id != null) {
277               idsPresentInN[vuln.unified.vulnerability_id] = true;
278             }
279           }
280         }
281       }
282     }
283
284     if (resolutionDate == null) return null;
285
286     def indexDate = resolutionDate.substring(0,10).replace('-', '.');
287
288     // iterate open ids from historical data
289     def open_buckets =
290       ctx.payload.open_vuln_details.aggregations.nested_vulns.open_only.unique_open_ids.buckets;
291
292     for (def b : open_buckets) {
293       def vulnId = b.key;
294       if (idsPresentInN.containsKey(vulnId)) continue;
295
296       // get sample parent doc for this vuln id
```

## ANNEXE C. DÉTAILS DE L'AUTOMATISATION DU SUIVI DE L'OBSERVABILITÉ (CHAPITRE 7)

```
294 def sampleHit = b.back_to_root.oldest_open_hit.hits.hits[0];
295 def parent = sampleHit._source;
296
297 // find vuln object inside parent.vulnerabilities
298 def targetVuln = null;
299 if (parent.vulnerabilities != null) {
300     for (def v : parent.vulnerabilities) {
301         if (v.unified?.vulnerability_id == vulnId) { targetVuln = v; break; }
302     }
303 }
304 if (targetVuln == null) continue;
305
306 def unified = targetVuln.unified;
307 def firstSeen = unified?.first_seen;
308 def resol_date = unified?.resolution_date != null ?
    unified.resolution_date : null;
309
310 // respect your original condition: firstSeen != null AND (resol_date ==
    '1970...' OR resol_date == null)
311 if (firstSeen != null && (resol_date == '1970-01-01T00:00:00Z' ||
    resol_date == null)) {
312     // compute mttr hours
313     def resolvedMs =
        java.time.ZonedDateTime.parse(resolutionDate).toInstant().toEpochMilli();
314
315     int offsetIndex = firstSeen.indexOf('+');
316     if (offsetIndex == -1) offsetIndex = firstSeen.indexOf('-');
317
318     def firstSeenDate;
319     if (offsetIndex != -1 && firstSeen.length() - offsetIndex == 5) {
320         def corrected = firstSeen.substring(0, offsetIndex+3) + ':' +
            firstSeen.substring(offsetIndex+3);
321         firstSeenDate = java.time.ZonedDateTime.parse(corrected);
322     } else {
323         firstSeenDate = java.time.ZonedDateTime.parse(firstSeen);
324     }
325     def firstSeenMs = firstSeenDate.toInstant().toEpochMilli();
326     long diff = resolvedMs - firstSeenMs;
327     double mttrHours = (diff > 0) ? (double) diff / 3600000.0 : 0.0;
328
329     // build resolved_parent entry (keeps minimal structured data)
330     resolved_parent.add([
331         "vulnerability_id": vulnId,
332         "service": parent.service != null ? parent.service : null,
333         "tool": parent.tool != null ? parent.tool : null,
334         "unified": [
335             "vulnerability_id": vulnId,
```

## ANNEXE C. DÉTAILS DE L'AUTOMATISATION DU SUIVI DE L'OBSERVABILITÉ (CHAPITRE 7)

```
336     "status": "closed",
337     "resolution_date": resolutionDate,
338     "first_seen": firstSeen,
339     "mttr_hours": mttrHours,
340     "category": unified?.category
341   ]
342   });
343
344   // build flattened resolved finding (fields prefixed resolved.*)
345   def flat = [:];
346   // resolved.vulnerability_id
347   flat['resolved.vulnerability_id'] = vulnId;
348   // resolved.service.*
349   if (parent.service != null) {
350     for (def e : parent.service.entrySet()) {
351       flat['resolved.service.' + e.getKey()] = e.getValue();
352     }
353   }
354   // resolved.tool.*
355   if (parent.tool != null) {
356     for (def e : parent.tool.entrySet()) {
357       flat['resolved.tool.' + e.getKey()] = e.getValue();
358     }
359   }
360   // resolved.metadata.* if exists
361   if (parent.metadata != null) {
362     for (def e : parent.metadata.entrySet()) {
363       flat['resolved.metadata.' + e.getKey()] = e.getValue();
364     }
365   }
366   // unified flattened under resolved.unified.*
367   if (unified != null) {
368     for (def e : unified.entrySet()) {
369       flat['resolved.unified.' + e.getKey()] = e.getValue();
370     }
371   }
372   // override critical resolved fields
373   flat['resolved.unified.status'] = 'closed';
374   flat['resolved.unified.resolution_date'] = resolutionDate;
375   flat['resolved.unified.mttr_hours'] = mttrHours;
376   flat['resolved.unified.first_seen'] = firstSeen;
377   flat['@timestamp'] = resolutionDate;
378   flat['doc_type'] = 'resolved_finding';
379   flat['report_id'] = parent.report_id != null ? parent.report_id : null;
380
381   resolved_findings.add(flat);
382 }
```

## ANNEXE C. DÉTAILS DE L'AUTOMATISATION DU SUIVI DE L'OBSERVABILITÉ (CHAPITRE 7)

```
383 }
384
385 return [
386   "index_date": indexDate,
387   "resolutionDate": resolutionDate,
388   "resolved_parent": resolved_parent,
389   "resolved_findings": resolved_findings
390 ];
391 "",
392   "lang": "painless"
393 }
394 }
395 }
```

Listing C.1 – Code complet du Watcher de résolution des vulnérabilités

### C.2 Code de la Requête MTTR

```
1 {
2   "size": 0,
3   "query": {
4     "term": {
5       "doc_type": "resolved_finding"
6     }
7   },
8   "aggs": {
9     "mttr_by_day": {
10      "date_histogram": {
11        "field": "@timestamp",
12        "calendar_interval": "1d",
13        "format": "yyyy-MM-dd"
14      },
15      "aggs": {
16        "average_mttr_hours": {
17          "avg": {
18            "field": "resolved.unified.mttr_hours"
19          }
20        }
21      }
22    }
23  }
24 }
```

Listing C.2 – Requête Elasticsearch pour le calcul du MTTR moyen quotidien



## **ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES**

**www.esprit.tn - E-mail : [contact@esprit.tn](mailto:contact@esprit.tn)**

**Siège Social :** 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

**Annexe :** Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454